

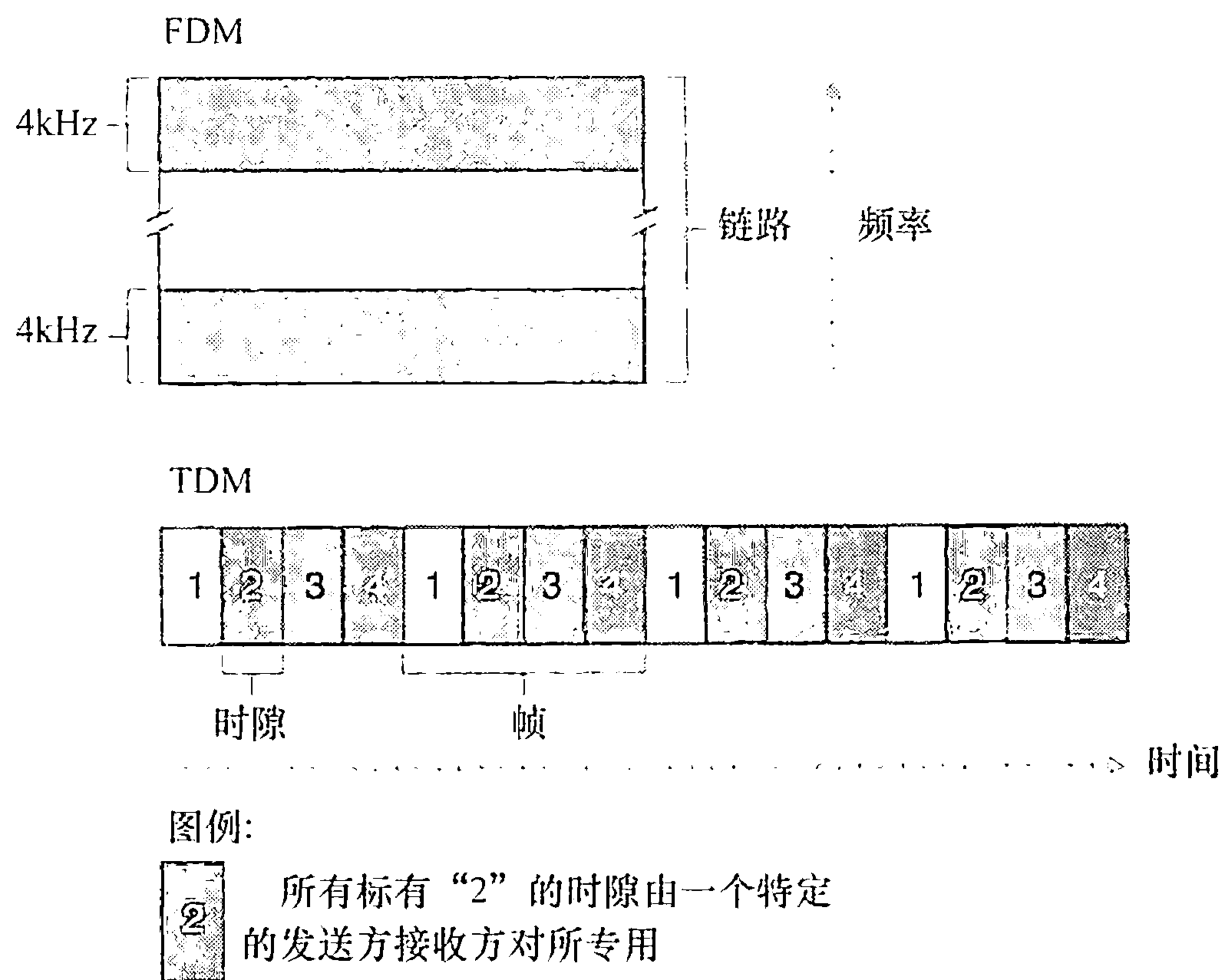


图 1-14 对于 FDM，每条电路连续地得到部分带宽。对于 TDM，每条电路在短时间间隔（即时隙）中周期性地得到所有带宽

在结束讨论电路交换之前，我们讨论一个用数字表示的例子，它更能说明问题的实质。考虑从主机 A 到主机 B 经一个电路交换网络需要多长时间发送一个 640 000 比特的文件。假如在该网络中所有链路使用 24 时隙的 TDM，具有 1.536Mbps 的比特速率。同时假定在主机 A 能够开始传输该文件之前，需要 500ms 创建一条端到端电路。它需要多长时间才能发送该文件？每条链路具有的传输速率是 $1.536\text{Mbps}/24 = 64\text{kbps}$ ，因此传输该文件需要 $(640\text{kb})/(64\text{kbps}) = 10\text{s}$ 。对于这个 10s，再加上电路创建时间，这样就需要 10.5s 发送该文件。值得注意的是，该传输时间与链路数量无关：端到端电路不管是通过一条链路还是 100 条链路，传输时间都将是 10s。（实际的端到端时延还包括传播时延，参见 1.4 节。）

2. 分组交换与电路交换的对比

在描述了电路交换和分组交换之后，我们来对比一下这两者。分组交换的批评者经常争辩说，分组交换不适合实时服务（例如，电话和视频会议），因为它的端到端时延是可变的和不可预测的（主要是因为排队时延的变动和不可预测所致）。分组交换的支持者却争辩道：①它提供了比电路交换更好的带宽共享；②它比电路交换更简单，更有效，实现成本更低。分组交换与电路交换之争的有趣讨论参见 [Molinero-Fernandez 2002]。概括而言，嫌餐馆预订麻烦的人宁可要分组交换而不愿意要电路交换。

分组交换为什么更有效呢？我们看一个简单的例子。假定多个用户共享一条 1Mbps 链路，再假定每个用户活跃周期是变化的，某用户时而以 100kbps 恒定速率产生数据，时而静止——这时用户不产生数据。进一步假定该用户仅有 10% 的时间活跃（余下的 90% 的时间空闲下来喝咖啡）。对于电路交换，在所有的时间内必须为每个用户预留 100kbps。例如，对于电路交换的 TDM，如果一个 1s 的帧被划分为 10 个时隙，每个时隙为 100ms，则每帧将为每个用户分配一个时隙。

因此，该电路交换链路仅能支持 10（= 1Mbps/100kbps）个并发的用户。对于分组交

换，一个特定用户活跃的概率是 0.1（即 10%）。如果有 35 个用户，有 11 或更多个并发活跃用户的概率大约是 0.0004。（课后习题 P8 概述如何得到这个概率值。）当有 10 个或更少并发用户（以概率 0.9996 发生）时，到达的聚合数据率小于或等于该链路的输出速率 1Mbps。因此，当有 10 个或更少个活跃用户时，通过该链路的分组流基本上没有时延，这与电路交换的情况一样。当同时活跃用户超过 10 个时，则分组的聚合到达率超过该链路的输出容量，则输出队列将开始变长。（一直增长到聚合输入速率重新低于 1Mbps，此后该队列长度才会减少。）因为在本例子中同时活跃用户超过 10 个的概率极小，分组交换差不多总是提供了与电路交换相同的性能，并且允许在用户数量是其 3 倍时情况也是如此。

我们现在考虑第二个简单的例子。假定有 10 个用户，某个用户突然产生 1000 个 1000 比特的分组，而其他用户则保持静默，不产生分组。在每帧具有 10 个时隙并且每个时隙包含 1000 比特的 TDM 电路交换情况下，活跃用户仅能使用每帧中的一个时隙来传输数据，而每个帧中剩余的 9 个时隙保持空闲。该活跃用户传输完所有 10^6 比特数据需要 10s 的时间。在分组交换情况下，活跃用户能够连续地以 1Mbps 的全部链路速率发送其分组，因为没有其他用户产生分组与该活跃用户的分组进行复用。在此情况下，该活跃用户的所有数据将在 1s 内发送完毕。

上面的例子从两个方面表明了分组交换的性能能够优于电路交换的性能。这些例子也强调了两种形式的在多个数据流之间共享链路传输速率的关键差异。电路交换不考虑需求，而预先分配了传输链路的使用，这使得已分配而并不需要的链路时间未被利用。另一方面，分组交换按需分配链路使用。链路传输能力将在所有用户之间逐分组地被共享，这些用户有分组需要在链路上传输。

虽然分组交换和电路交换在今天的电信网络中都是普遍采用的方式，但趋势无疑是朝着分组交换方向发展。甚至许多今天的电路交换电话网正在缓慢地向分组交换迁移。特别是，电话网经常在昂贵的海外电话部分使用分组交换。

1.3.3 网络的网络

我们在前面看到，端系统（PC、智能手机、Web 服务器、电子邮件服务器等）经过一个接入 ISP 与因特网相连。该接入 ISP 能够提供有线或无线连接，使用了包括 DSL、电缆、FTTH、WiFi 和蜂窝等多种接入技术。值得注意的是，接入 ISP 不必是本地电信局或电缆公司，相反，它能够是如大学（为学生、教职员工和从业人员提供因特网接入）或公司（为其雇员提供接入）这样的单位。但为端用户和内容提供商提供与接入 ISP 的连接仅解决了连接难题中的很小一部分，因为因特网是由数以亿计的用户构成的。要解决这个难题，接入 ISP 自身必须互联。通过创建网络的网络可以做到这一点，理解这个短语是理解因特网的关键。

年复一年，构成因特网的“网络的网络”已经演化成为一个非常复杂的结构。这种演化很大部分是由经济和国家策略驱动的，而不是由性能考虑驱动的。为了理解今天的因特网网络结构，我们来逐步递进建造一系列网络结构，其中的每个新结构都更好地接近我们现有的复杂因特网。回顾前面互联接入 ISP 的目标，是使所有端系统能够彼此发送分组。一种幼稚的方法是使每个接入 ISP 直接与每个其他接入 ISP 连接。当然，这样的网状设计对于接入 ISP 费用太高，因为这将要求每个接入 ISP 与世界上数十万个其他接入 ISP 有一条单独的通信链路。

我们的第一个网络结构即网络结构 1，用单一的全球承载 ISP 互联所有接入 ISP。我们假想的全球承载 ISP 是一个由路由器和通信链路构成的网络，该网络不仅跨越全球，而且至少具有一个路由器靠近数十万接入 ISP 中的每一个。当然，对于全球承载 ISP，建造这样一个大规模的网络将耗资巨大。为了有利可图，自然要向每个连接的接入 ISP 收费，其价格反映（并不一定正比于）一个接入 ISP 经过全球 ISP 交换的流量大小。因为接入 ISP 向全球承载 ISP 付费，故接入 ISP 被认为是**客户**（customer），而全球承载 ISP 被认为是**提供商**（provider）。

如果某个公司建立并运行了一个可赢利的全球承载 ISP，其他公司建立自己的全球承载 ISP 并与最初的全球承载 ISP 竞争则是一件自然的事。这导致了网络结构 2，它由数十万接入 ISP 和多个全球承载 ISP 组成。接入 ISP 无疑喜欢网络结构 2 胜过喜欢网络结构 1，因为它们现在能够根据价格和服务的函数，在多个竞争的全球承载提供商之间进行选择。然而，值得注意的是，这些全球承载 ISP 之间必须是互联的：不然的话，与某个全球承载 ISP 连接的接入 ISP 将不能与连接到其他全球承载 ISP 的接入 ISP 通信。

刚才描述的网络结构 2 是一种两层的等级结构，其中全球承载提供商位于顶层，而接入 ISP 位于底层。这假设了全球承载 ISP 不仅能够接近每个接入 ISP，而且发现经济上也希望这样做。现实中，尽管某些 ISP 确实具有令人印象深刻的全球覆盖，并且确实直接与许多接入 ISP 连接，但世界上没有 ISP 是存在于每个城市中的。相反，在任何给定的区域，可能有一个**区域 ISP**（regional ISP），区域中的接入 ISP 与之连接。每个区域 ISP 则与**第一层 ISP**（tier-1 ISP）连接。第一层 ISP 类似于我们假想的全球承载 ISP；尽管第一层 ISP 不是在世界每个城市中都存在，但它确实存在。有大约十几个第一层 ISP，包括 Level 3 通信、AT&T、Sprint 和 NTT。有趣的是，没有组织正式认可第一层状态。俗话说：如果必须问你是否是一个组织的成员，你可能不是。

返回到网络的网络，不仅有多个竞争的第一层 ISP，而且在一个区域可能有多个竞争的区域 ISP。在这样的等级结构中，每个接入 ISP 向区域 ISP 支付其连接费用，并且每个区域 ISP 向它连接的第一层 ISP 支付费用。（一个接入 ISP 也能直接与第一层 ISP 连接，这样它就向第一层 ISP 付费。）因此，在这个等级结构的每层，有**客户-提供商**关系。值得注意的是，第一层 ISP 不向任何人付费，因为它们位于该等级结构的顶部。使事情更为复杂的是，在某些区域，可能有较大的区域 ISP（可能跨越整个国家），区域中较小的区域 ISP 与之相连，较大的区域 ISP 则与第一层 ISP 连接。例如，在中国，每个城市有接入 ISP，它们与省级 ISP 连接，省级 ISP 又与国家级 ISP 连接，国家级 ISP 最终与第一层 ISP 连接 [Tian 2012]。这个多层等级结构仍然仅仅是今天因特网的粗略近似，我们称它为网络结构 3。

为了建造一个与今天因特网更为相似的网络，我们必须在等级结构的网络结构 3 上增加存在点（Point of Presence, PoP）、多宿、对等和因特网交换点（Internet exchange point, IXP）。PoP 存在于等级结构的所有层次，但底层（接入 ISP）等级除外。一个 PoP 只是提供商网络中的一台或多台路由器（在相同位置）群组，其中客户 ISP 能够与提供商 ISP 连接。对于要与提供商 PoP 连接的客户网络，它能从第三方通信提供商租用高速链路直接将它的路由器之一连接到位于该 PoP 的一台路由器。任何 ISP（除了第一层 ISP）可以选择为**多宿**（multi-home），即可以与两个或更多提供商 ISP 连接。例如，一个接入 ISP 可能与两个区域 ISP 多宿，或者可以与两个区域 ISP 多宿，也可以与多个第一层 ISP 多宿。当一个 ISP 多宿时，即使它的提供商之一出现故障，它仍然能够继续发送和接收分组。

正如我们刚才学习的，客户 ISP 向它们的提供商 ISP 付费以获得全球因特网互联能力。客户 ISP 支付给提供商 ISP 的费用数额反映了它通过提供商交换的流量。为了减少这些费用，位于相同等级结构层次的邻近一对 ISP 能够对等（peer），这就是说，能够直接将它们的网络连到一起，使它们之间的所有流量经直接连接而不是通过上游的中间 ISP 传输。当两个 ISP 对等时，通常不进行结算，即任一个 ISP 不向其对等付费。如前面提到的那样，第一层 ISP 也与另一个第一层 ISP 对等，它们之间无结算。对于对等和客户 - 提供商关系可读性的讨论，参见 [Van der Berg 2008]。沿着这些相同路线，第三方公司创建一个因特网交换点（Internet Exchange Point, IXP）（通常在一个有自己的交换机的独立建筑物中），IXP 是一个汇合点，多个 ISP 能够在这里共同对等。在今天的因特网中有大约 300 个 IXP [Augustin 2009]。我们称这个系统为生态系统——由接入 ISP、区域 ISP、第一层 ISP、PoP、多宿、对等和 IXP 组成，这个系统作为网络结构 4。

我们现在最终到达了网络结构 5，它描述了 2012 年的因特网。在图 1-15 中显示了网络结构 5，它通过在网络结构 4 顶部增加内容提供商网络（content provider network）构建而成。谷歌是当前这样的内容提供商网络的一个突出例子。在本书写作之时，谷歌估计有 30 ~ 50 个数据中心部署在北美、欧洲、亚洲、南美和澳大利亚。其中的某些数据中心容纳了超过十万台的服务器，而另一些数据中心则较小，仅容纳数百台服务器。谷歌数据中心都经过专用的 TCP/IP 网络互联，该网络跨越全球，但仍然独立于公共因特网。重要的是，谷歌专用网络仅承载出入谷歌服务器主机的流量。如图 1-15 所示，谷歌专用网络通过与较低层 ISP 对等（无结算）尝试“绕过”因特网的较高层，采用的方式可以是直接与它们连接，或者在 IXP 处与它们连接 [Labovitz 2010]。然而，因为许多接入 ISP 通过第一层网络的承载仍能到达，所以谷歌网络也与第一层 ISP 连接，并就与它们交换的流量向这些 ISP 付费。通过创建自己的网络，内容提供商不仅减少了向顶层 ISP 支付的费用，而且对其服务最终如何交付给端用户有了更多的控制。谷歌的网络基础设施在 7.2.4 节中进行了详细描述。

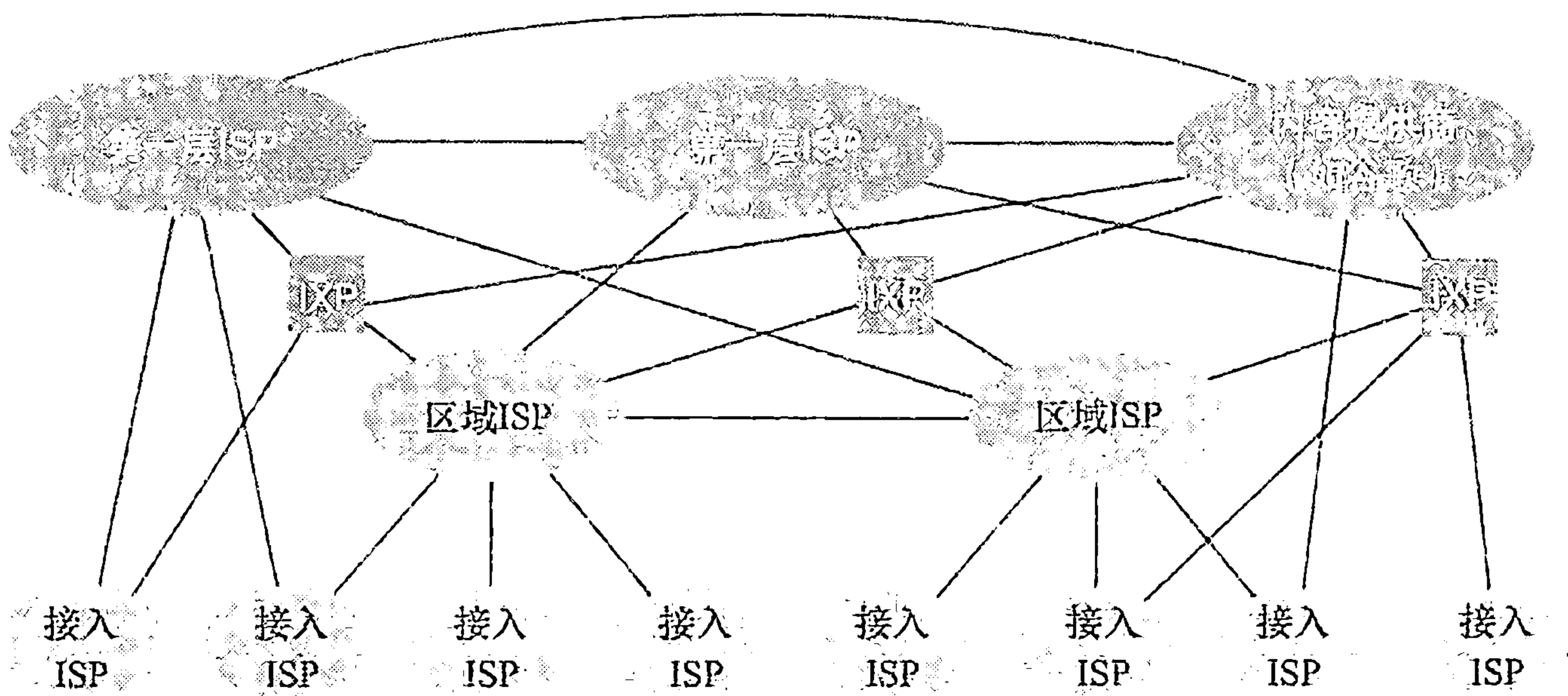


图 1-15 ISP 的互联

总结一下，今天的因特网是一个网络的网络，其结构复杂，由十多个第一层 ISP 和数十万个较低层 ISP 组成。ISP 覆盖的区域有所不同，有些跨越多个大洲和大洋，有些限于很小的地理区域。较低层的 ISP 与较高层的 ISP 相连，较高层 ISP 彼此互联。用户和内容提供商是较低层 ISP 的客户，较低层 ISP 是较高层 ISP 的客户。近年来，主要的内容提供商也已经创建自己的网络，直接在可能的地方与较低层 ISP 互联。

1.4 分组交换网中的时延、丢包和吞吐量

回想在 1.1 节中我们讲过，因特网能够看成是一种为运行在端系统上的分布式应用提供服务的基础设施。在理想情况下，我们希望因特网服务能够在任意两个端系统之间瞬间移动我们想要的大量数据而没有任何数据丢失。然而，这是一个极高的目标，实践中难以达到。与之相反，计算机网络必定要限制在端系统之间的吞吐量（每秒能够传送的数据量），在端系统之间引入时延，而且实际上能够丢失分组。一方面，现实世界的物理定律引入的时延、丢包并限制吞吐量是不幸的。而另一方面，因为计算机网络存在这些问题，围绕如何去处理这些问题有许多令人着迷的话题，多得足以开设一门有关计算机网络方面的课程，可以做上千篇博士论文！在本节中，我们将开始研究和量化计算机网络中的时延、丢包和吞吐量等问题。

1.4.1 分组交换网中的时延概述

前面讲过，分组从一台主机（源）出发，通过一系列路由器传输，在另一台主机（目的地）中结束它的历程。当分组从一个结点（主机或路由器）沿着这条路径到后继结点（主机或路由器），该分组在沿途的每个结点经受了几种不同类型的时延。这些时延最为重要的是结点处理时延（nodal processing delay）、排队时延（queuing delay）、传输时延（transmission delay）和传播时延（propagation delay），这些时延总体累加起来是结点总时延（total nodal delay）。许多因特网应用，如搜索、Web 浏览、电子邮件、地图、即时讯息和 IP 语音，它们的性能受网络时延的影响都很大。为了深入理解分组交换和计算机网络，我们必须理解这些时延的性质和重要性。

时延的类型

我们来探讨一下图 1-16 环境中的这些时延。作为源和目的地之间的端到端路径的一部分，一个分组从上游结点通过路由器 A 向路由器 B 发送。我们的目标是在路由器 A 刻画出结点时延。值得注意的是，路由器 A 具有通往路由器 B 的出链路。该链路前面有一个队列（也称为缓存）。当该分组从上游结点到达路由器 A 时，路由器 A 检查该分组的首部以决定该分组的适当出链路，并将该分组导向该链路。在这个例子中，对该分组的出链路是通向路由器 B 的那条链路。仅当在该链路没有其他分组正在传输并且没有其他分组排在该队列前面时，才能在这条链路上传输该分组；如果该链路当前正繁忙或有其他分组已经在该链路上排队，则新到达的分组则将参与排队。

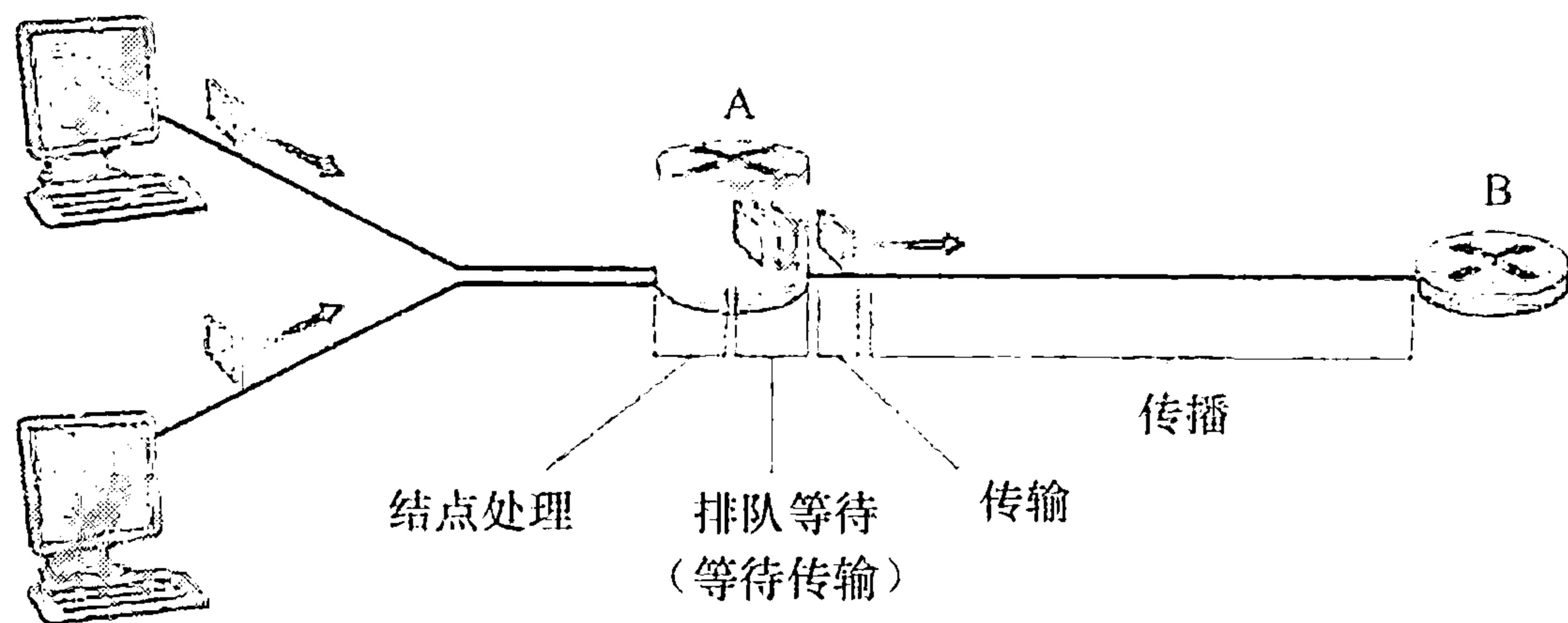


图 1-16 路由器 A 的结点时延

(1) 处理时延

检查分组首部和决定将该分组导向何处所需要的时间是处理时延的一部分。处理时延也能包括其他因素，如检查比特级别的差错所需要的时间，该差错出现在从上游结点向路由器 A 传输这些分组比特的过程中。高速路由器的处理时延通常是微秒或更低的数量级。在这种结点处理之后，路由器将该分组引向通往路由器 B 链路之前的队列。（在第 4 章中，我们将研究路由器运行的细节。）

(2) 排队时延

在队列中，当分组在链路上等待传输时，它经受排队时延。一个特定分组的排队时延长度将取决于先期到达的正在排队等待向链路传输的分组数量。如果该队列是空的，并且当前没有其他分组正在传输，则该分组的排队时延为 0。另一方面，如果流量很大，并且许多其他分组也在等待传输，该排队时延将很长。我们将很快看到，到达分组期待发现的分组数量是到达该队列的流量的强度和性质的函数。实际的排队时延可以是毫秒到微秒量级。

(3) 传输时延

假定分组以先到先服务方式传输，这在分组交换网中是常见的方式，仅当所有已经到达的分组被传输后，才能传输刚到达的分组。用 L 比特表示该分组的长度，用 R bps（即 b/s）表示从路由器 A 到路由器 B 的链路传输速率。例如，对于一条 10Mbps 的以太网链路，速率 $R = 10\text{Mbps}$ ；对于 100Mbps 的以太网链路，速率 $R = 100\text{Mbps}$ 。传输时延是 L/R 。这是将所有分组的比特推（传输）向链路所需要的时间。实际的传输时延通常在毫秒到微秒量级。

(4) 传播时延

一旦一个比特被推向链路，该比特需要向路由器 B 传播。从该链路的起点到路由器 B 传播所需要的时间是传播时延。该比特以该链路的传播速率传播。该传播速率取决于该链路的物理媒体（即光纤、双绞铜线等），其速率范围是 $2 \times 10^8 \sim 3 \times 10^8 \text{ m/s}$ ，这等于或略小于光速。该传播时延等于两台路由器之间的距离除以传播速率。即传播时延是 d/s ，其中 d 是路由器 A 和路由器 B 之间的距离， s 是该链路的传播速率。一旦该分组的最后一个比特传播到结点 B，该比特及前面的所有比特被存储于路由器 B。整个过程将随着路由器 B 执行转发而持续下去。在广域网中，传播时延为毫秒量级。

(5) 传输时延和传播时延的比较

计算机网络领域的新手有时难以理解传输时延和传播时延之间的差异。该差异是微妙而重要的。传输时延是路由器将分组推出所需要的时间，它是分组长度和链路传输速率的函数，而与两台路由器之间的距离无关。另一方面，传播时延是一个比特从一台路由器向另一台路由器传播所需要的时间，它是两台路由器之间距离的函数，而与分组长度或链路传输速率无关。

一个类比可以阐明传输时延和传播时延的概念。考虑一条公路每 100km 有一个收费站，如图 i-17 所示。可认为收费站间的公路段是链路，收费站是路由器。假定汽车以 100km/h 的速度在该公路上行驶（即传播）（即当一辆汽车离开一个收费站时，它立即加速到 100km/h 并在收费站间维持该速度）。假定这时有 10 辆汽车的车队在行驶，并且这 10 辆汽车以固定的顺序互相跟随。可以认为每辆汽车是一个比特，该车队是一个分组。同时假定每个收费站以每辆车 12s 的速度服务（即传输）一辆汽车，由于时间是深夜，因

此该车队是公路上唯一一批汽车。最后，假定无论该车队的第一辆汽车何时到达收费站，它在入口处等待，直到其他 9 辆汽车到达并整队依次前行。（因此，整个车队在它能够“转发”之前，必须存储在收费站。）收费站将整个车队推向公路所需要的时间是（10 辆车）/（5 辆车/min）= 2min。该时间类比于一台路由器中的传输时延。因此，一辆汽车从一个收费站出口行驶到下一个收费站所需要的时间是 $100\text{h}/(100\text{km/h}) = 1\text{h}$ 。这个时间类比于传播时延。因此，从该车队存储在收费站前到该车队存储在下一个收费站前的时间是“传输时延”和“传播时间”总和，在本例中为 62min。

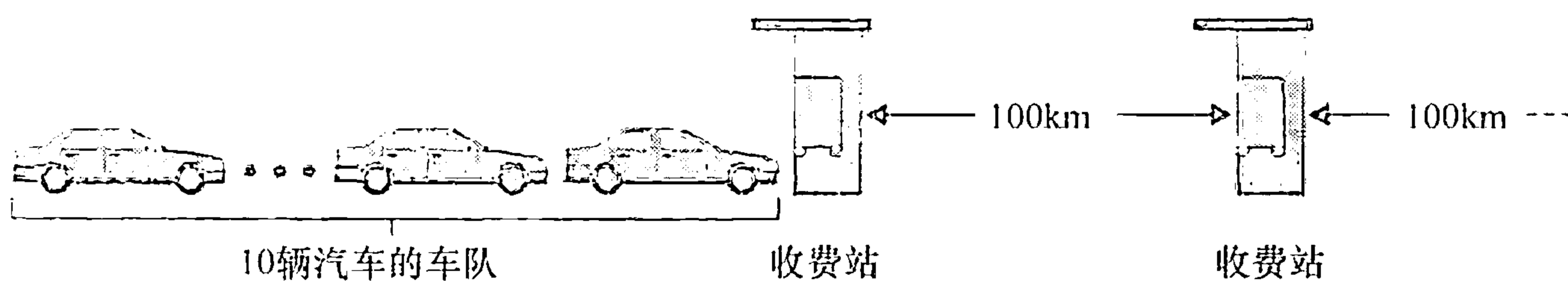


图 1-17 车队的类比

我们更深入地探讨一下这个类比。如果收费站对车队的服务时间大于汽车在收费站之间行驶的时间，将会发生什么情况呢？例如，假定现在汽车是以 1000km/h 的速率行驶，收费站是以每分钟一辆汽车的速率为汽车服务。则汽车在两个收费站之间的行驶时间延是 6min，收费站为车队服务的时间是 10min。在此情况下，在该车队中的最后几辆汽车离开第一个收费站之前，该车队中前面的几辆汽车将会达到第二个收费站。这种情况在分组交换网中也会发生，一个分组中的前几个比特到达了一台路由器，而该分组中许多余下的比特仍然在前面的路由器中等待传输。

如果说一图胜千言的话，则一个动画必定胜百万言。与本书配套的 Web 网站提供了一个交互式 Java 小程序，它很好地展现及对比了传输时延和传播时延。我们极力推荐读者访问该 Java 小程序。[Smith 2009] 也提供了可读性很好的有关传播、排队和传输时延的讨论。

如果我们令 d_{proc} 、 d_{queue} 、 d_{trans} 和 d_{prop} 分别表示处理时延、排队时延、传输时延和传播时延，则结点的总时延由下式给定：

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

这些时延成分所起的作用可能变化很大。例如， d_{prop} 对于连接两台位于同一个大学校园的路由器的链路而言可能是微不足道的（例如，几个微秒）；然而， d_{proc} 对于由同步卫星链路互联的两台路由器来说是几百毫秒，能够成为 d_{nodal} 中的主要成分。类似地， d_{trans} 的影响能够是微不足道的，也能是很大的。它的影响通常对于 10Mbps 和更高的传输速率（例如，对于 LAN）的信道而言是微不足道的；然而，对于通过低速拨号调制解调器链路发送的长因特网分组而言，可能是数百毫秒。处理时延 d_{proc} 通常是微不足道的；然而，它对一台路由器的最大吞吐量有重要影响，最大吞吐量是一台路由器能够转发分组的最大速率。

1.4.2 排队时延和丢包

结点时延的最为复杂和有趣的成分是排队时延 d_{queue} 。事实上，排队时延在计算机网络中的重要程度及人们对它感兴趣的程度，从发表的数以千计的论文和大量专著的情况可见一斑 [Bersekas 1991; Daigle 1991; Kleinrock 1975, 1976; Ross 1995]。我们这里仅给出

有关排队时延的总体的、直觉的讨论；求知欲强的读者可能要浏览某些书籍（或者最终写有关这方面的博士论文）。与其他 3 项时延（即 d_{proc} 、 d_{trans} 和 d_{prop} ）不同的是，排队时延对不同的分组可能是不同的。例如，如果 10 个分组同时到达空队列，传输的第一个分组没有排队时延，而传输的最后一个分组将经受相对大的排队时延（这时它要等待其他 9 个分组被传输）。因此，当表征排队时延时，人们通常使用统计量测度，如平均排队时延、排队时延的方差和排队时延超过某些特定值的概率。

什么时候排队时延大，什么时候又不大呢？该问题的答案很大程度取决于流量到达该队列的速率、链路的传输速率和到达流量的性质，即流量是周期性到达还是以突发形式到达。为了更深入地领会某些要点，令 a 表示分组到达队列的平均速率（ a 的单位是分组/秒，即 pkt/s）。前面讲过 R 是传输速率，即从队列中推出比特的速率（以 bps 即 b/s 为单位）。为了简单起见，也假定所有分组都是由 L 比特组成的。则比特到达队列的平均速率是 La bps。最后，假定该队列非常大，因此它基本能容纳无限数量的比特。比率 La/R 被称为流量强度（traffic intensity），它在估计排队时延的范围方面经常起着重要的作用。如果 $La/R > 1$ ，则比特到达队列的平均速率超过从该队列传出去的速率。在这种不幸的情况下，该队列趋向于无界增加，并且排队时延将趋向无穷大！因此，流量工程中的一条金科玉律是：设计系统时流量强度不能大于 1。

现在考虑 $La/R \leq 1$ 时的情况。这时，到达流量的性质影响排队时延。例如，如果分组周期性到达，即每 L/R 秒到达一个分组，则每个分组将到达一个空队列中，不会有排队时延。在另一方面，如果分组以突发形式到达而不是周期性到达，则有很大的平均排队时延。例如，假定每 $(L/R)N$ 秒同时到达 N 个分组。则传输的第一个分组没有排队时延；传输的第二个分组就有 L/R 秒的排队时延；更为一般地，第 n 个传输的分组具有 $(n-1)L/R$ 秒的排队时延。我们将该例子中的计算平均排队时延的问题留给读者作为练习。

以上描述周期性到达的两个例子有些学术味。到达队列的过程通常是随机的，即到达并不遵循任何模式，分组之间的时间间隔是随机的。在这种更为真实的情况下，量 La/R 通常不足以全面地表征时延的统计量。不过，直观地理解排队时延的范围很有用。特别是，如果流量强度接近于 0，则几乎没有分组到达并且到达间隔很大，那么到达的分组将不可能在队列中发现别的分组。因此，平均排队时延将接近 0。在另一方面，当流量强度接近 1 时，将存在到达率超过传输能力的时间间隔（由于分组到达率的波动），在这些时段中将形成队列。无论如何，随着流量强度接近 1，平均排队长度变得越来越长。平均排队时延与流量强度的定性关系如图 1-18 所示。

图 1-18 的一个重要方面是这样的事实：随着流量强度接近于 1，平均排队时延迅速增加。该强度少量的增加将导致时延大得多的增加。也许你在公路上经历过这种事。如果经常在通常拥塞的公路上驾驶，这条路经常拥塞的事实意味着它的流量强度接近于 1。如果某些事件引起一个甚至稍微大于平常量的流量，经受的时延就可能很大。

为了真实地感受一下排队时延的情况，我们再次鼓励你访问本书的 Web 网站，该网站提供了一个有

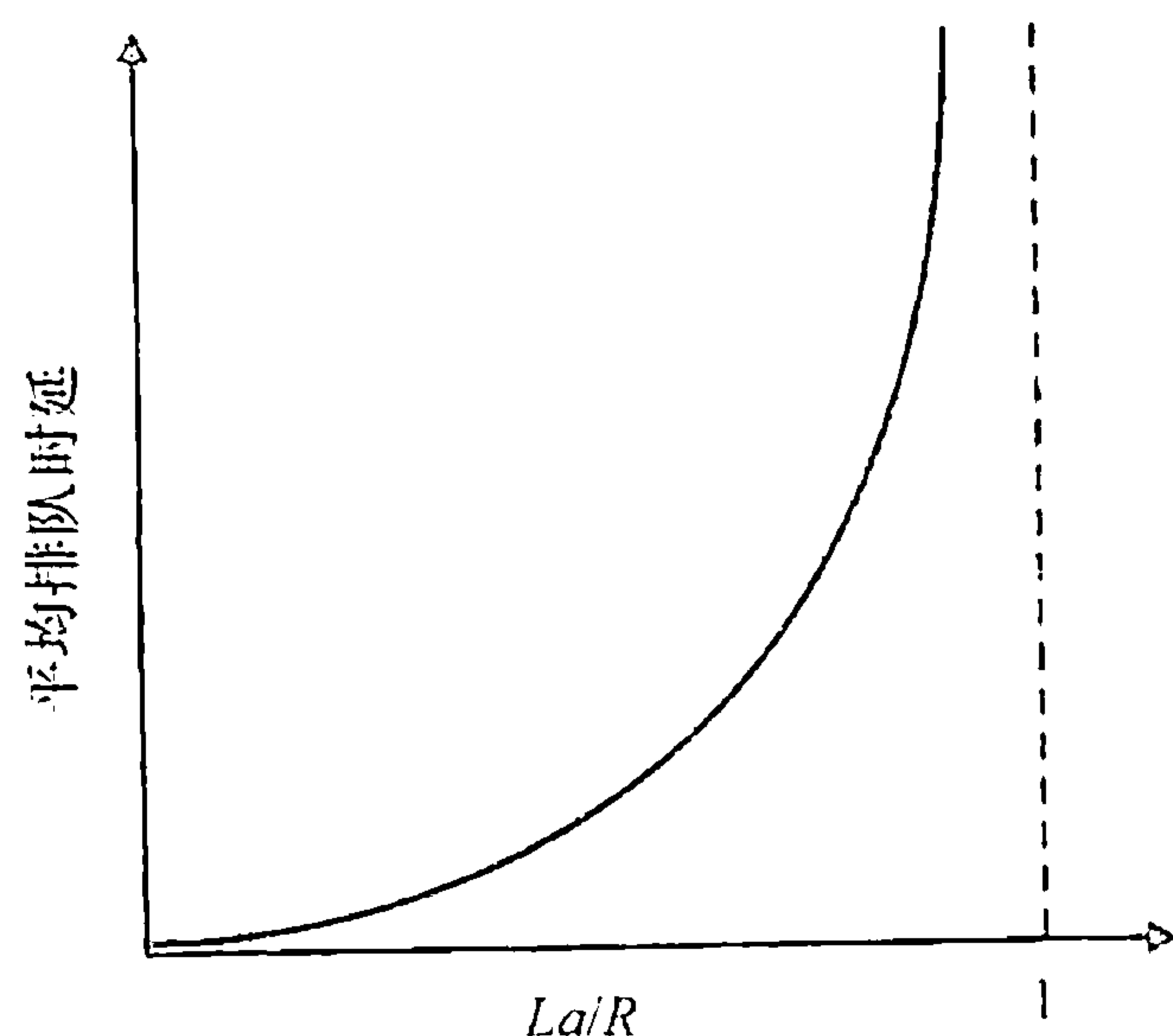


图 1-18 平均排队时延与流量强度的关系

关队列的交互式 Java 小程序。如果你将分组到达率设置得足够大，使流量强度超过 1，那么将看到经过一段时间后，队列慢慢地建立起来。

丢包

在上述讨论中，我们已经假设队列能够容纳无穷多的分组。在现实中，一条链路前的队列只有有限的容量，尽管排队容量极大地依赖于路由器设计和成本。因为该排队容量是有限的，随着流量强度接近 1，排队时延并不实际趋向无穷大。相反，到达的分组将发现一个满的队列。由于没有地方存储这个分组，路由器将丢弃（drop）该分组，即该分组将会丢失（lost）。当流量强度大于 1 时，队列中的这种溢出也能够在用于队列的 Java 小程序中看到。

从端系统的角度看，上述丢包现象看起来是一个分组已经传输到网络核心，但它绝不会从网络发送到目的地。分组丢失的份额随着流量强度增加而增加。因此，一个结点的性能常常不仅根据时延来度量，而且根据分组丢失的概率来度量。正如我们将在后面各章中讨论的那样，丢失的分组可能基于端到端的原则重传，以确保所有的数据最终从源传送到目的地。

1.4.3 端到端时延

前面的讨论一直集中在结点时延上，即在单台路由器上的时延。我们现在考虑从源到目的地的总时延。为了能够理解这个概念，假定在源主机和目的主机之间有 $N-1$ 台路由器。我们还要假设该网络此时是无拥塞的（因此排队时延是微不足道的），在每台路由器和源主机上的处理时延是 d_{proc} ，每台路由器和源主机的输出速率是 R bps，每条链路的传播时延是 d_{prop} 。结点时延累加起来，得到端到端时延：

$$d_{\text{end-end}} = N(d_{\text{proc}} + d_{\text{trans}} + d_{\text{prop}}) \quad (1-2)$$

式中再次有 $d_{\text{trans}} = L/R$ ，其中 L 是分组长度。值得注意的是，式（1-2）是式（1-1）的一般形式，式（1-1）没有考虑处理时延和传播时延。在各结点具有不同的时延和每个结点存在平均排队时延的情况下，需要对式（1-2）进行一般化处理。我们将有关工作留给读者。

1. Traceroute

为了对计算机网络中的时延有一个第一手认识，我们可以利用 Traceroute 程序。Traceroute 是一个简单的程序，它能够在任何因特网主机上运行。当用户指定一个目的主机名字时，源主机中的该程序朝着该目的地发送多个特殊的分组。当这些分组向着目的地传送时，它们通过一系列路由器。当路由器接收到这些特殊分组之一时，它向源回送一个短报文。该报文包括该路由器名字和地址。

更具体的是，假定在源和目的地之间有 $N-1$ 台路由器。则源将向网络发送 N 个特殊的分组，其中每个分组地址指向最终目的地。这 N 个特殊分组标识为从 1 到 N ，第一个分组标识为 1，最后的分组标识为 N 。当第 n 台路由器接收到标识为 n 的第 n 个分组时，该路由器不是向它的目的地转发该分组，而是向源回送一个报文。当目的主机接收第 N 个分组时，它也会向源返回一个报文。该源记录了从它发送一个分组到它接收到对应返回报文所经受的时间；它也记录了返回该报文的路由器（或目的地主机）的名字和地址。以这种方式，源能够重建分组从源到目的地所采用的路由，并且该源能够决定到所有中间路由器的往返时延。Traceroute 实际上对刚才描述的实验重复了 3 次，因此该源实际上向目的地

发送了 $3 \times N$ 个分组。RFC 1393 详细地描述了 Traceroute。

这里有一个 Traceroute 程序输出的例子，其中追踪的路由从源主机 gaia.cs.umass.edu（位于马萨诸塞大学）到 cis.poly.edu（位于布鲁克林的理工大学）。输出有 6 列：第一列是前面描述的 n 值，即沿着路径上的路由器编号；第二列是路由器的名字；第三列是路由器地址（格式为 xxx.xxx.xxx.xxx）；最后 3 列是 3 次实验的往返时延。如果源从任何给定的路由器接收到少于 3 条报文（由于网络中的丢包），Traceroute 在该路由器号码后面放一个星号，并向那台路由器报告少于 3 次往返时间。

```

1 cs-gw (128.119.240.254) 1.009 ms 0.899 ms 0.993 ms
2 128.119.3.154 (128.119.3.154) 0.931 ms 0.441 ms 0.651 ms
3 border4-rt-gi-1-3.gw.umass.edu (128.119.2.194) 1.032 ms 0.484 ms 0.451 ms
4 acr1-ge-2-1-0.Boston.cw.net (208.172.51.129) 10.006 ms 8.150 ms 8.460 ms
5 agr4-loopback.NewYork.cw.net (206.24.194.104) 12.272 ms 14.344 ms 13.267 ms
6 acr2-loopback.NewYork.cw.net (206.24.194.62) 13.225 ms 12.292 ms 12.148 ms
7 pos10-2.core2.NewYork1.Level3.net (209.244.160.133) 12.218 ms 11.823 ms 11.793 ms
8 gige9-1-52.hsipaccess1.NewYork1.Level3.net (64.159.17.39) 13.081 ms 11.556 ms 13.297 ms
9 p0-0.polyu.bbnplanet.net (4.25.109.122) 12.716 ms 13.052 ms 12.786 ms
10 cis.poly.edu (128.238.32.126) 14.080 ms 13.035 ms 12.802 ms

```

在上述跟踪中，在源和目的之间有 9 台路由器。这些路由器中的多数有一个名字，所有都有地址。例如，路由器 3 的名字是 border4-rt-gi-1-3.gw.umass.edu，它的地址是 128.119.2.194。看看为这台路由器提供的数据，可以看到在源和路由器之间的往返时延：3 次试验中的第一次是 1.03ms，后继两次试验的往返时延是 0.48ms 和 0.45ms。这些往返时延包括刚才讨论的所有时延，即包括传输时延、传播时延、路由器处理时延和排队时延。因为该排队时延随时间变化，分组 n 发送到路由器 n 的往返时延实际上能够比分组 $n+1$ 发送到路由器 $n+1$ 的往返时延更长。的确，我们在上述例子中观察到了这种现象：到路由器 6 的时延比到路由器 7 的更大！

你想自己试试 Traceroute 程序吗？我们极力推荐你访问 <http://www.traceroute.org>，它的 Web 界面提供了有关路由跟踪的广泛的源列表。你选择一个源，并为任何目的地提供主机名，该 Traceroute 程序则会完成所有工作。有许多为 Traceroute 提供图形化界面的免费软件程序，其中我们喜爱的一个程序是 PingPlotter [PingPlotter 2012]。

2. 端系统、应用程序和其他时延

除了处理时延、传输时延和传播时延，端系统中还有其他一些重要时延。例如，作为它的协议的一部分，希望向共享媒体（例如在 WiFi 或电缆调制解调器情况下）传输分组的端系统可以有意地延迟它的传输以与其他端系统共享媒体；我们将在第 5 章中详细地考虑这样的一些协议。另一个重要的时延是媒体分组化时延，这种时延出现在经 IP 语音 (VoIP) 应用中。在 VoIP 中，发送方在向因特网传递分组之前必须首先用编码的数字化语音填充一个分组。这种填充一个分组的时间称为分组化时延，它可能较大，并能够影响用户感受到的 VoIP 呼叫的质量。这个问题将在本章结束的课后作业中进一步探讨。

1.4.4 计算机网络中的吞吐量

除了时延和丢包，计算机网络中另一个必不可少的性能测度是端到端吞吐量。为了定义吞吐量，考虑从主机 A 到主机 B 跨越计算机网络传送一个大文件。例如，也许是从一个 P2P 文件共享系统中的一个对等方向另一个对等方传送一个大视频片段。在任何时间瞬间的瞬时吞吐量 (instantaneous throughput) 是主机 B 接收到该文件的速率（以 bps 计）。（许

多应用程序包括许多文件共享系统，在下载期间其用户界面显示了其瞬时吞吐量，也许你以前已经观察过它！）如果该文件由 F 比特组成，主机 B 接收到所有 F 比特用去 T 秒，则文件传送的平均吞吐量（average throughput）是 F/T bps。对于某些应用程序如因特网电话，希望具有低时延和在某个阈值之上的一致的瞬时吞吐量。例如，对某些因特网电话是超过 24kbps，对某些实时视频应用程序是超过 256kbps。对于其他应用程序，包括涉及文件传送的那些应用程序，时延不是至关重要的，但是希望具有尽可能高的吞吐量。

为了进一步深入理解吞吐量这个重要概念，我们考虑几个例子。图 1-19a 显示了服务器和客户这两个端系统，它们由两条通信链路和一台路由器相连。考虑从服务器传送一个文件到客户的吞吐量。令 R_s 表示服务器与路由器之间的链路速率； R_c 表示路由器与客户之间的链路速率。假定在整个网络中只有从这台服务器到那台客户的比特在传送。在这种理想的情况下，我们现在问该服务器到客户的吞吐量是多少？为了回答这个问题，我们可以想象比特是流体，通信链路是管道。显然，这台服务器不能以快于 R_s bps 的速率通过其链路注入比特；这台路由器也不能以快于 R_c bps 的速率转发比特。如果 $R_s < R_c$ ，则由该服务器注入的比特将顺畅地通过路由器“流动”，并以速率 R_s bps 到达客户，给定了 R_s bps 的吞吐量。在另一方面，如果 $R_c < R_s$ ，则该路由器将不能够以接收它们那样快的速率来转发比特。在这种情况下，比特将以速率 R_c 离开该路由器，从而得到端到端吞吐量 R_c 。（还要注意的，如果比特继续以速率 R_s 到达该路由器，继续以 R_c 离开路由器的话，在该路由器中等待传输给客户的积压比特将不断增加，这是一种非常不希望的情况！）因此，对于这种简单的两链路的网络，其吞吐量是 $\min\{R_c, R_s\}$ ，这就是说，它是瓶颈链路（bottleneck link）的传输速率。在决定了吞吐量之后，我们现在近似地得到从服务器到客户传输一个 F 比特的大文件所需要的时间是 $F/\min\{R_c, R_s\}$ 。对于一个特定的例子，假定你正在下载一个 $F = 32 \times 10^6$ 比特的 MP3 文件，服务器具有 $R_s = 2\text{Mbps}$ 的传输速率，并且你有一条 $R_c = 1\text{Mbps}$ 的接入链路。则传输该文件所需的时间是 32 秒。当然，这些吞吐量和传送时间的表达式仅是近似的，因为它们并没有考虑分组层次和协议的问题。

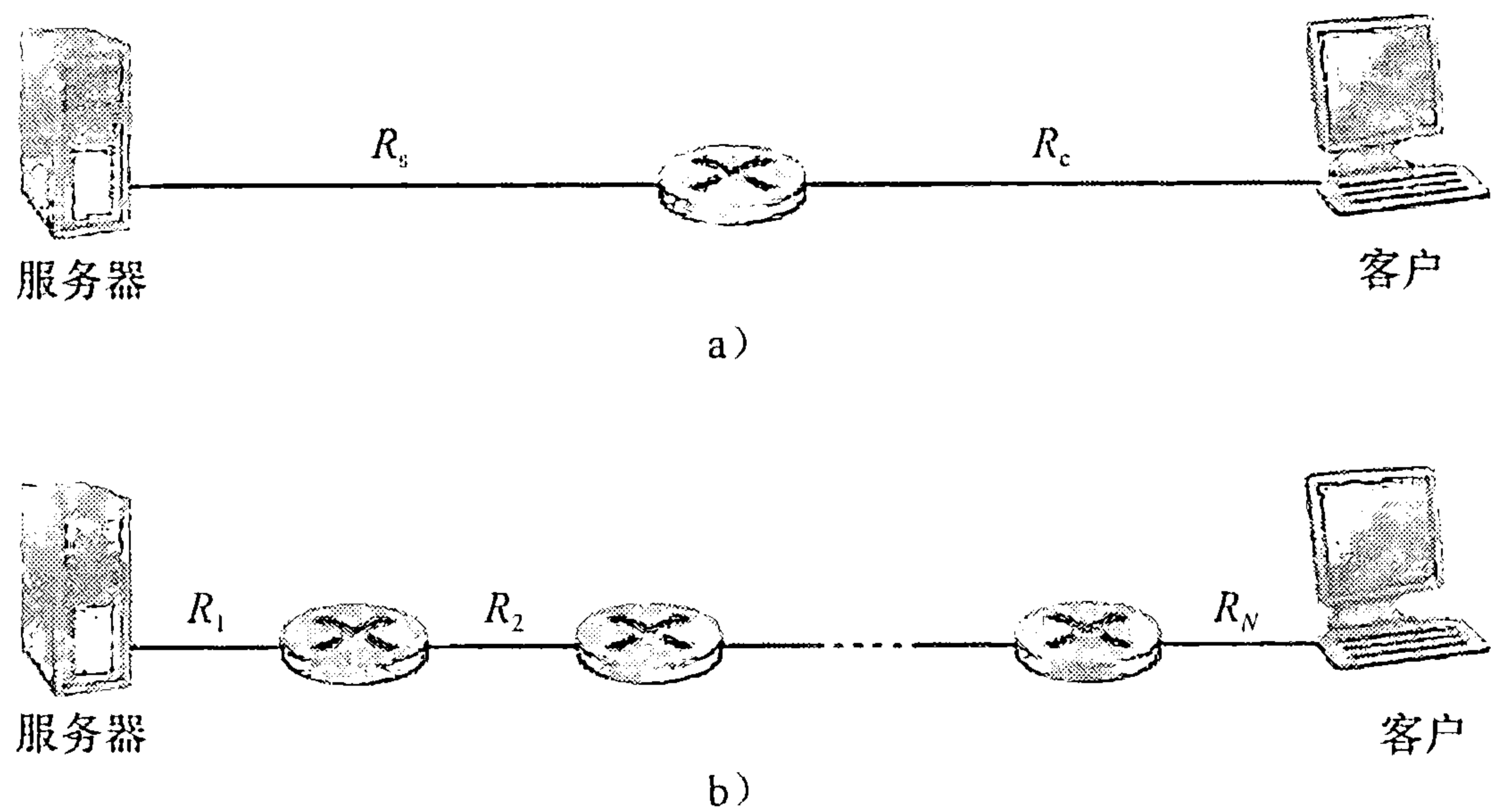


图 1-19 一个文件从服务器传送到客户的吞吐量

图 1-19b 此时显示了在服务器和客户之间具有 N 条链路的一个网络，这 N 条链路的传输速率分别是 $R_1, R_2, \dots, R_N$ 。应用与对两条链路网络的分析相同的方法，我们发现从服务器到客户的文件传输的吞吐量是 $\min\{R_1, R_2, \dots, R_N\}$ ，这同样仍是沿着服务器和客户之间路径的瓶颈链路的速率。

现在考虑由当前因特网所引发的另一个例子。图 1-20a 显示了与一个计算机网络相连的两个端系统：一台服务器和一个客户。考虑从服务器向客户的文件传送的吞吐量。服务器以速率为 R_s 的接入速率与网络相连，且客户以速率为 R_c 的接入速率与网络相连。现在假定在计算机网络核心中的所有链路具有非常高的传输速率，即该速率比 R_s 和 R_c 要高得多。目前因特网的核心的确过度装备了高速率的链路，从而很少出现拥塞。同时假定在整个网络中发送的比特都是从该服务器到该客户。在这个例子中，因为计算机网络的核心就像一个宽大的管子，所以比特从服务器向目的地的流动速率仍是 R_s 和 R_c 中的最小者，即吞吐量 $= \min\{R_s, R_c\}$ 。因此，在今天因特网中对吞吐量的限制因素通常是接入网。

作为最后一个例子，考虑图 1-20b，其中有 10 台服务器和 10 个客户与某计算机网络核心相连。在这个例子中，同时发生 10 个下载，涉及 10 个客户 - 服务器对。假定这 10 个下载是网络中当时的唯一流量。如该图所示，在核心中有一条所有 10 个下载通过的链路。将这条链路 R 的传输速率表示为 R 。假定所有服务器接入链路具有相同的速率 R_s ，所有客户接入链路具有相同的速率 R_c ，并且核心中除了速率为 R 的一条共同链路之外的所有链路传输速率都比 R_s 、 R_c 和 R 大得多。现在我们要问，这种下载的吞吐量是多少？显然，如果该公共链路的速率 R 很大，比如说比 R_s 和 R_c 大 100 倍，则每个下载的吞吐量将仍然是 $\min\{R_s, R_c\}$ 。但是如果该公共链路的速率与 R_s 和 R_c 有相同量级会怎样呢？在这种情况下其吞吐量将是多少呢？让我们观察一个特定的例子。假定 $R_s = 2\text{Mbps}$ ， $R_c = 1\text{Mbps}$ ， $R = 5\text{Mbps}$ ，并且公共链路在 10 个下载之间平等划分它的传输速率。这时每个下载的瓶颈不再位于接入网中，而是位于核心中的共享链路了，该瓶颈仅能为每个下载提供 500kbps 的吞吐量。因此每个下载的端到端吞吐量现在减少到 500kbps。

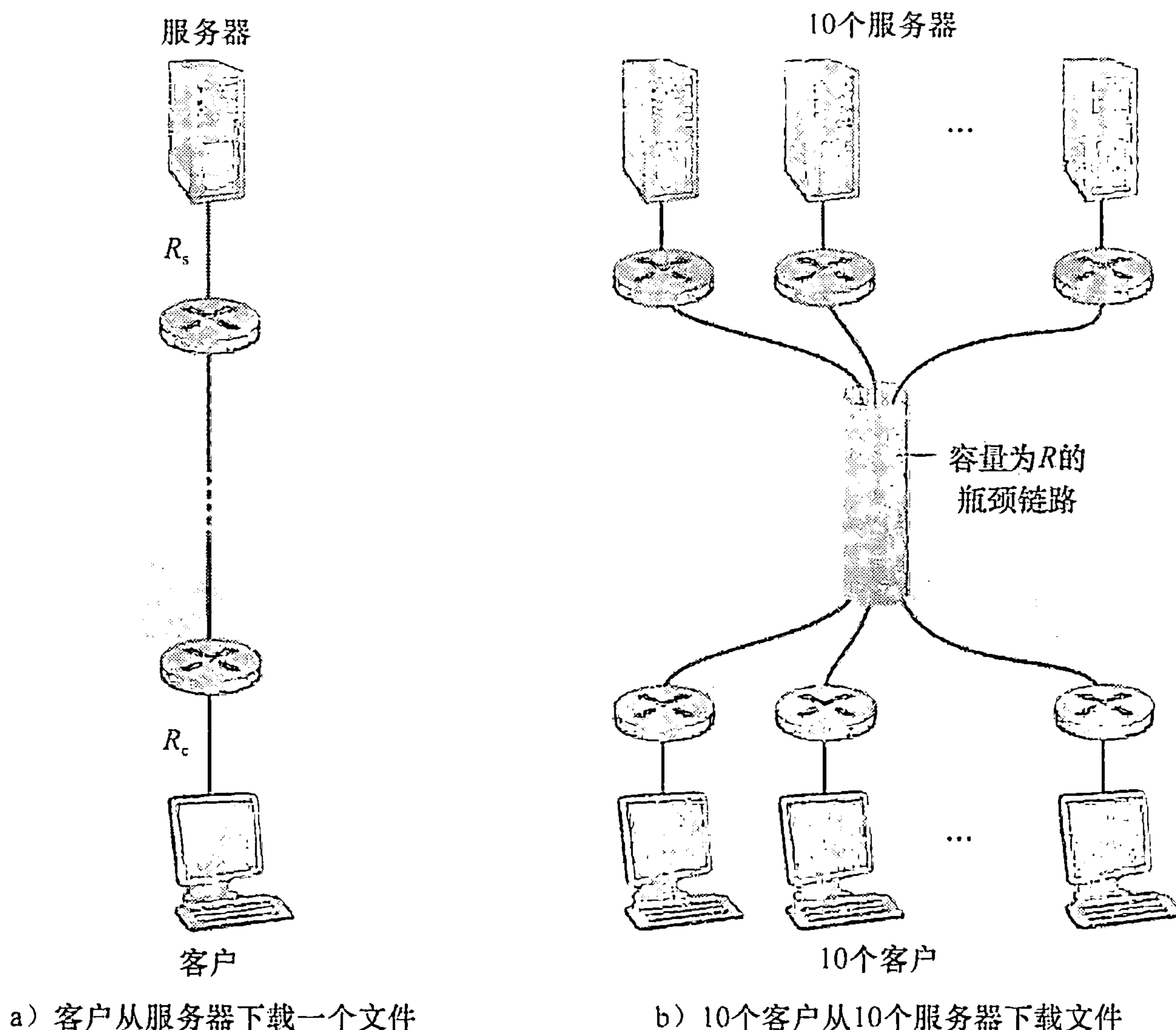


图 1-20 端到端吞吐量

图 1-19 和图 1-20 中的例子说明吞吐量取决于数据流过的链路的传输速率。我们看到当没有其他干扰流量时，其吞吐量能够近似为沿着源和目的地之间路径的最小传输速率。图 1-20b 中的例子更一般地说明了吞吐量不仅取决于沿着路径的传输速率，而且取决于干扰流量。特别是，如果许多其他的数据流也通过这条链路流动，一条具有高传输速率的链路仍然可能成为文件传输的瓶颈链路。我们将在课后习题中和后继章节中更仔细地研究计算机网络中的吞吐量。

1.5 协议层次及其服务模型

从我们到目前的讨论来看，因特网是一个极为复杂的系统。我们已经看到，因特网有许多部分：大量的应用程序和协议、各种类型的端系统、分组交换机和各种类型的链路级媒体。面对这种巨大的复杂性，存在着组织网络体系结构的希望吗？或者至少存在着我们对网络体系结构进行讨论的希望吗？幸运的是，对这两个问题的回答都是肯定的。

1.5.1 分层的体系结构

在试图组织我们关于因特网体系结构的想法之前，先看一个人类社会与之类比的例子。实际上，在日常生活中我们一直都与复杂系统打交道。想象一下有人请你描述航线系统的情况吧。你怎样用一个结构来描述这样一个复杂的系统？该系统具有票务代理、行李检查、登机口人员、飞行员、飞机、空中航行控制和世界范围的导航系统。描述这种系统的一种方式，是描述当你乘某个航班时，你（或其他人替你）要采取的一系列动作。你要购买机票，托运行李，去登机口，并最终登上这次航班。该飞机起飞，飞行到目的地。当飞机着陆后，你从登机口离机并认领行李。如果这次行程不理想，你向票务机构投诉这次航班（你的努力一无所获）。图 1-21 显示了相关情况。

我们已经能从这里看出与计算机网络的某些类似：航空公司把你从源送到目的地；而分组被从因特网中的源主机送到目的主机。但这不是我们寻求的完全的类似。我们在图 1-21 中寻找某些结构。观察图 1-21，我们注意到在每一端都有票务功能；还对已经检票的乘客有行李功能，对已经检票并已经检查过行李的乘客有登机功能。对于那些已经通过登机的乘客（即已经经过检票、行李检查和通过登机的乘客），有起飞和着陆的功能，并且在飞行中，有飞机按预定路线飞行的功能。这提示我们能够以水平的方式看待这些功能，如图 1-22 所示。

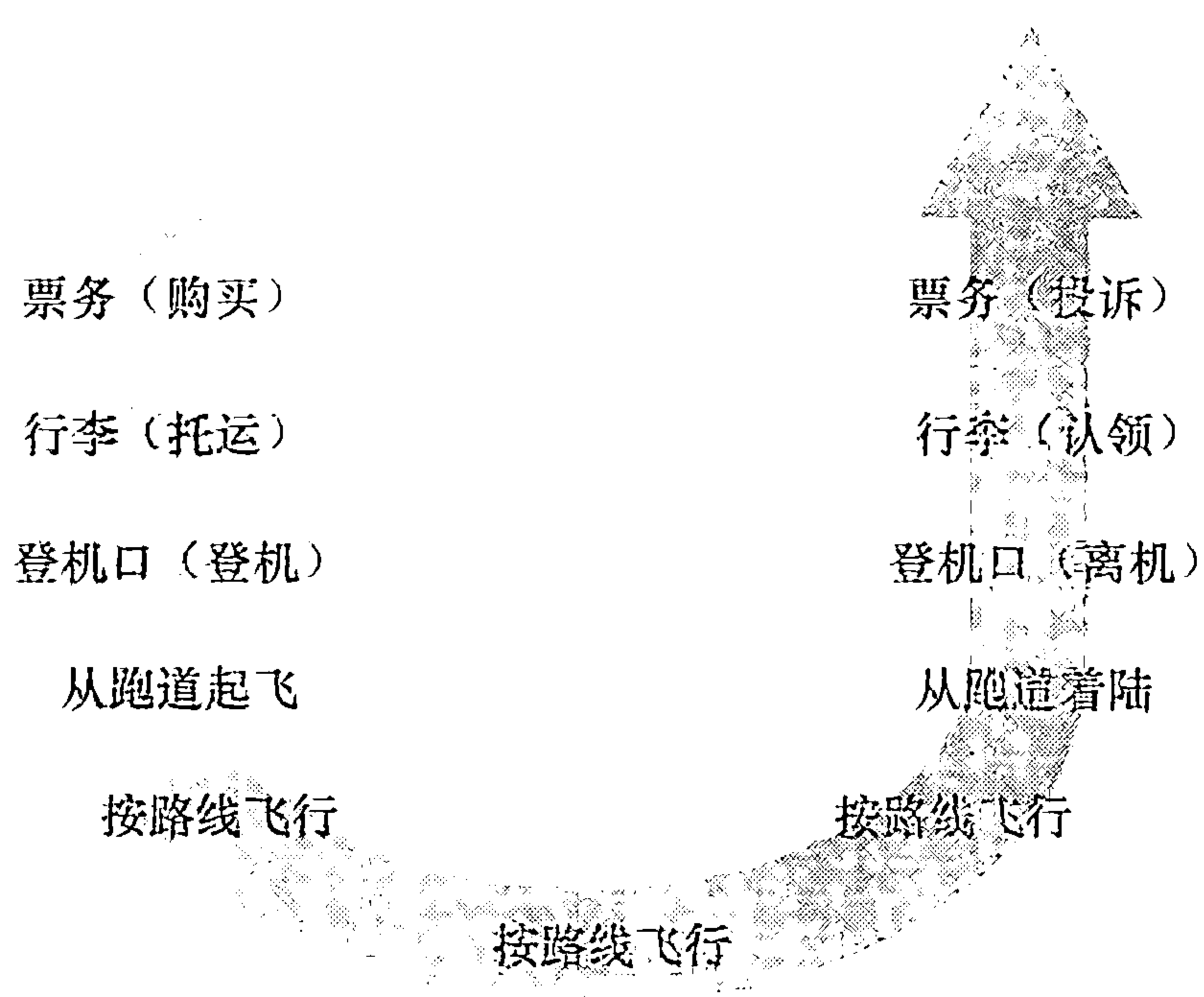


图 1-21 乘飞机旅行的一系列动作

图 1-22 将航线功能划分为一些层次，提供了我们能够讨论航线旅行的框架。值得注意的是每个层次与其下面的层次结合在一起，实现了某些功能、服务。在票务层及以下，完成了一个人的航线柜台到航线柜台的转移。在行李层及以下，完成了人和行李的行李托运到行李认领的转移。值得注意的是行李层仅对已经完成票务的人提供服务。在登机口

层，完成了人和行李的离港登机口到到港登机口的转移。在起飞/着陆层，完成了一个人及其行李的跑道到跑道的转移。每个层次通过以下方式提供服务：①在这层中执行了某些动作（例如，在登机口层，某飞机的乘客登机 and 离机）；②使用直接下层的的服务（例如，在登机口层，使用起飞/着陆层的跑道到跑道的旅客转移服务）。

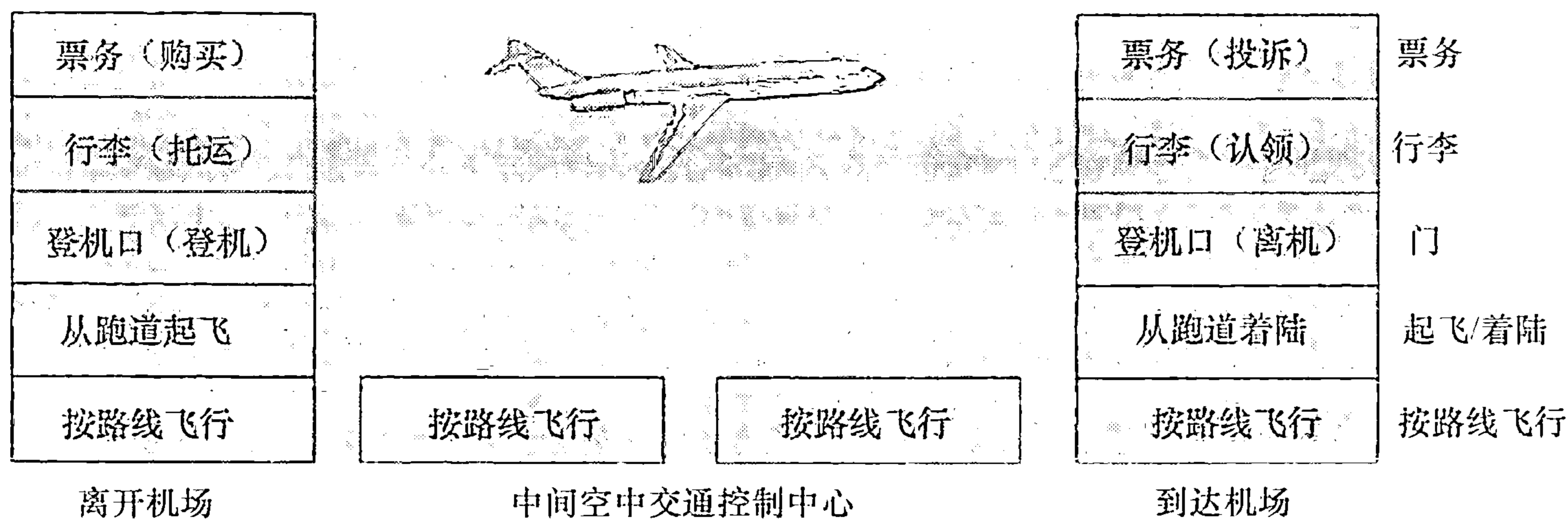


图 1-22 航线功能的水平分层

利用分层的体系结构，我们可以讨论一个定义良好的、大而复杂系统的特定部分。这种简化本身由于提供模块化而具有很高的价值，这使实现由层所提供的服务易于改变。只要该层对其上面的层提供相同的服务，并且使用来自下面层次的相同服务，当某层的实现变化时，该系统的其余部分保持不变。（值得注意的是，改变一个服务的实现与改变服务本身是极为不同的！）例如，如果登机口功能被改变了（例如让人们按身高登机和离机），航线系统的其余部分将保持不变，因为登机口仍然提供相同的功能（人们登机和离机）；改变后，它仅是以不同的方式实现了该功能。对于大而复杂且需要不断更新的系统，改变服务的实现而不影响该系统其他组件是分层的另一个重要优点。

1. 协议分层

我们对航线已经进行了充分讨论，现将注意力转向网络协议。为了给网络协议的设计提供一个结构，网络设计者以分层（layer）的方式组织协议以及实现这些协议的网络硬件和软件。每个协议属于这些层次之一，就像图 1-22 所示的航线体系结构中的每种功能属于某一层一样。我们再次关注某层向它的上一层提供的服务（service），即所谓一层的服务模型（service model）。就像前面航线例子中的情况一样，每层通过在该层中执行某些动作或使用直接下层的的服务来提供服务。例如，由第 n 层提供的服务可能包括报文从网络的一边到另一边的可靠传送。这可能是通过使用第 $n-1$ 层的边缘到边缘的不可靠报文传送服务，加上第 n 层的检测和重传丢失报文的功能来实现的。

一个协议层能够用软件、硬件或两者的结合来实现。诸如 HTTP 和 SMTP 这样的应用层协议几乎总是在端系统中用软件实现的，运输层协议也是如此。因为物理层和数据链路层负责处理跨越特定链路的通信，它们通常是实现在与给定链路相联系的网络接口卡（例如以太网或 WiFi 接口卡）中。网络层经常是硬件和软件实现的混合体。还要注意的，如同分层的航线体系结构中的功能分布在构成该系统的各机场和飞行控制中心中一样，一个第 n 层协议也分布在构成该网络的端系统、分组交换机和其他组件中。这就是说，第 n 层协议的不同部分常常位于这些网络组件的各部分中。

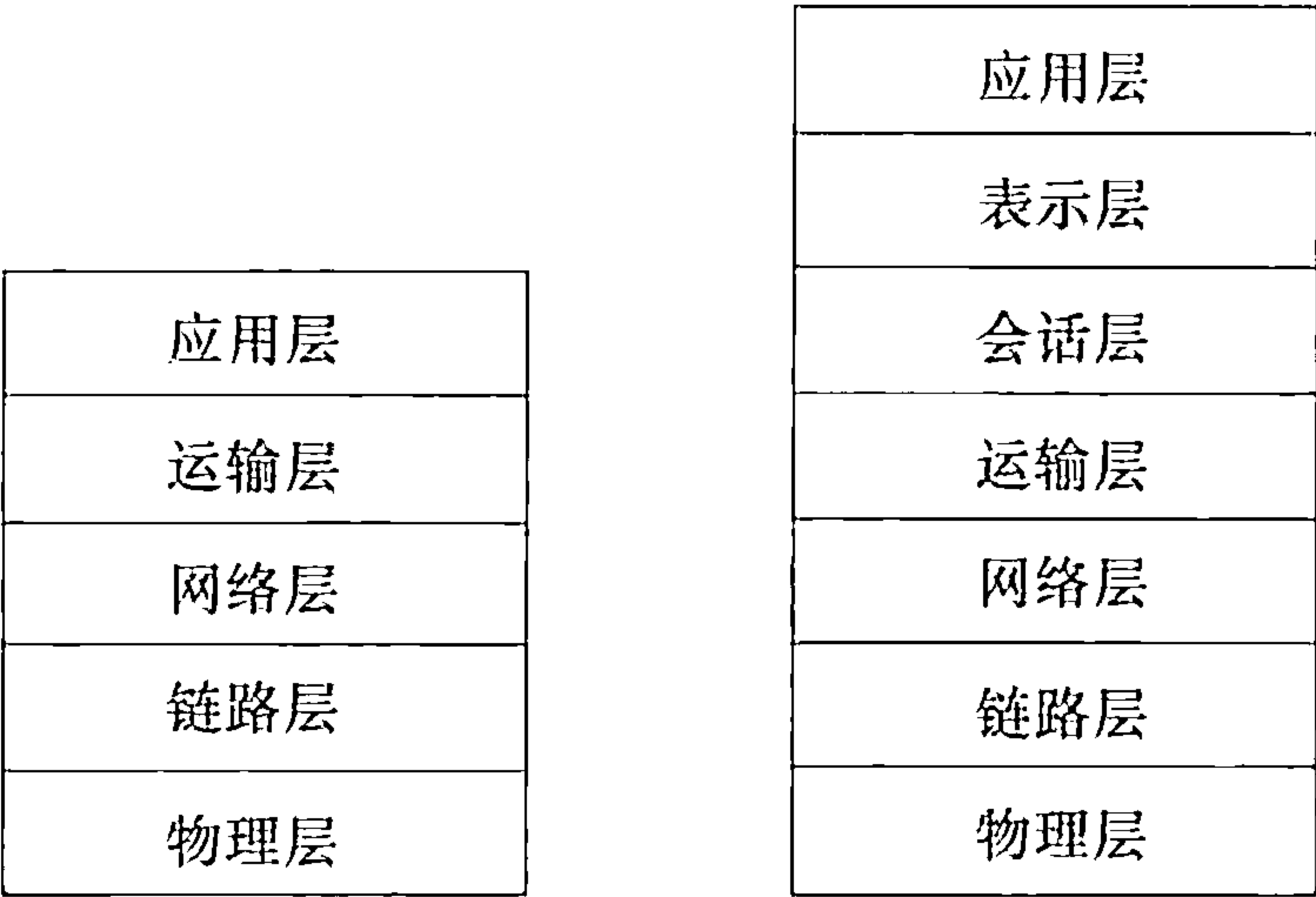
协议分层具有概念化和结构化的优点 [RFC 3439]。如我们看到的那样，分层提供了

一种结构化方式来讨论系统组件。模块化使更新系统组件更为容易。然而，需要提及的是，某些研究人员和联网工程师激烈地反对分层 [Wakeman 1992]。分层的一个潜在缺点是一层可能冗余较低层的功能。例如，许多协议栈在基于每段链路和基于端到端两种情况下，都提供了差错恢复。第二种潜在的缺点是某层的功能可能需要仅在其他某层才出现的信息（如时间戳值），这违反了层次分离的目标。

将这些综合起来，各层的所有协议被称为协议栈（protocol stack）。因特网的协议栈由 5 个层次组成：物理层、链路层、网络层、运输层和应用层（如图 1-23a 所示）。如果你查看本书目录，将会发现我们大致是以因特网协议栈的层次来组织本书的。我们采用了自顶向下方法（top-down approach），首先处理应用层，然后向下进行处理。

(1) 应用层

应用层是网络应用程序及它们的应用层协议存留的地方。因特网的应用层包括许多协议，例如 HTTP（它提供了 Web 文档的请求和传送），SMTP（它提供了电子邮件报文的传输）和 FTP（它提供两个端系统之间的文件传送）。我们将看到，某些网络功能，如将像 www.ietf.org 这样对人友好的端系统名字转换为 32 比特网络地址，也是借助于特定的应用层协议即域名系统（DNS）完成的。我们将在第 2 章中看到，创建并部署我们自己的新应用层协议是非常容易的。



a) 五层因特网协议栈 b) 七层ISO OSI参考模型

图 1-23 因特网协议栈和 OSI 参考模型

应用层协议分布在多个端系统上，一个端系统中的应用程序使用协议与另一个端系统中的应用程序交换信息的分组。我们把这种位于应用层的信息分组称为报文（message）。

(2) 运输层

因特网的运输层在应用程序端点之间传送应用层报文。在因特网中，有两个运输协议，即 TCP 和 UDP，利用其中的任一个都能运输应用层报文。TCP 向它的应用程序提供了面向连接的服务。这种服务包括了应用层报文向目的地的确保传递和流量控制（即发送方/接收方速率匹配）。TCP 也将长报文划分为短报文，并提供拥塞控制机制，因此当网络拥塞时，源抑制其传输速率。UDP 协议向它的应用程序提供无连接服务。这是一种不提供不必要服务的服 务，没有可靠性，没有流量控制，也没有拥塞控制。在本书中，我们把运输层分组称为报文段（segment）。

(3) 网络层

因特网的网络层负责将称为数据报（datagram）的网络层分组从一台主机移动到另一台主机。在一台源主机中的因特网运输层协议（TCP 或 UDP）向网络层递交运输层报文段和目的地址，就像你通过邮政服务寄信件时提供一个目的地址一样。

因特网的网络层包括著名的 IP 协议，该协议定义了 在数据报中的各个字段以及端系统和路由器如何作用于这些字段。仅有一个 IP 协议，所有具有网络层的因特网组件必须运行 IP 协议。因特网的网络层也包括决定路由的路由选择协议，它使得数据报根据该路由从源传输到目的地。因特网具有许多路由选择协议。如我们在 1.3 节所见，因特网是一个网络的网 络，在一个网络中，其网络管理者能够运行所希望的任何路由选择协议。尽管

网络层包括了 IP 协议和一些路由选择协议，但通常把它简单地称为 IP 层，这反映了 IP 是将因特网连接在一起的粘合剂这样的事实。

(4) 链路层

因特网的网络层通过源和目的地之间的一系列路由器路由数据报。为了将分组从一个结点（主机或路由器）移动到路径上的下一个结点，网络层必须依靠该链路层的服务。特别是在每个结点，网络层将数据报下传给链路层，链路层沿着路径将数据报传递给下一个结点。在下个结点，链路层将数据报上传给网络层。

由链路层提供的服务取决于应用于该链路的特定链路层协议。例如，某些协议基于链路提供可靠传递，从传输结点跨越一条链路到接收结点。值得注意的是，这种可靠的传递服务不同于 TCP 的可靠传递服务，TCP 提供从一个端系统到另一个端系统的可靠交付。链路层的例子包括以太网、WiFi 和电缆接入网的 DOCSIS 协议。因为数据报从源到目的地传送通常需要经过几条链路，一个数据报可能被沿途不同链路上的不同链路层协议处理。例如，一个数据报可能被一段链路上的以太网和下一段链路上的 PPP 所处理。网络层将受到来自每个不同的链路层协议的不同服务。在本书中，我们把链路层分组称为帧（frame）。

(5) 物理层

虽然链路层的任务是将整个帧从一个网络元素移动到邻近的网络元素，而物理层的任务是将该帧中的一个一个比特从一个结点移动到下一个结点。在这层中的协议仍然是链路相关的，并且进一步与该链路（例如，双绞铜线、单模光纤）的实际传输媒体相关。例如，以太网具有许多物理层协议：一个是关于双绞铜线的，另一个是关于同轴电缆的，还有一个是关于光纤的，等等。在每种场合中，跨越这些链路移动一个比特是以不同的方式进行的。

2. OSI 模型

详细地讨论过因特网协议栈后，我们应当提及它不是唯一的协议栈。特别是在 20 世纪 70 年代后期，国际标准化组织（ISO）提出计算机网络应组织为大约 7 层，称为开放系统互连（OSI）模型 [ISO 2012]。当那些要成为因特网协议的协议尚处于襁褓中，只是许多正在研发之中的不同协议族之一时，OSI 模型已经成形；事实上，初始 OSI 模型的发明者在创建该模型时心中并没有想到因特网。无论如何，自 20 世纪 70 年代开始，许多培训课程和大学课程围绕 7 层模型挑选有关 ISO 授权和组织的课程。因为它在网络教育的早期影响，该 7 层模型继续以某种方式存留在某些网络教科书和培训课程中。

显示在图 1-23b 中的 OSI 参考模型的 7 层是：应用层、表示层、会话层、运输层、网络层、数据链路层和物理层。这些层次中，5 层的功能大致与它们名字类似的因特网对应层的相同。所以，我们来考虑 OSI 参考模型中附加的两个层，即表示层和会话层。表示层的作用是使通信的应用程序能够解释交换数据的含义。这些服务包括数据压缩和数据加密（它们是自解释的）以及数据描述（如我们将在第 9 章所见，这使得应用程序不必担心在各台计算机中表示/存储的内部格式不同的问题）。会话层提供了数据交换定界和同步功能，包括了建立检查点和恢复方案的方法。

因特网缺少了在 OSI 参考模型中建立的两个层次，该事实引起了一些有趣的问题：这些层次提供的服务不重要吗？如果一个应用程序需要这些服务之一，将会怎样呢？因特网对这两个问题的回答是相同的：这留给应用程序开发者处理。应用程序开发者决定一个服

务是否是重要的，如果该服务重要，应用程序开发者就应该在应用程序中构建该功能。

1.5.2 封装

图 1-24 显示了这样一条物理路径：数据从发送端系统的协议栈向下，向上和向下经过中间的链路层交换机和路由器的协议栈，进而向上到达接收端系统的协议栈。如我们将在本书后面讨论的那样，路由器和链路层交换机都是分组交换机。与端系统类似，路由器和链路层交换机以多层次的方式组织它们的网络硬件和软件。而路由器和链路层交换机并不实现协议栈中的所有层次。如图 1-24 所示，链路层交换机实现了第一层和第二层；路由器实现了第一层到第三层。例如，这意味着因特网路由器能够实现 IP 协议（一种第三层协议），而链路层交换机则不能。我们将在后面看到，尽管链路层交换机不能识别 IP 地址，但它们能够识别第二层地址，如以太网地址。值得注意的是，主机实现了所有 5 个层次，这与因特网体系结构将它的复杂性放在网络边缘的观点是一致的。

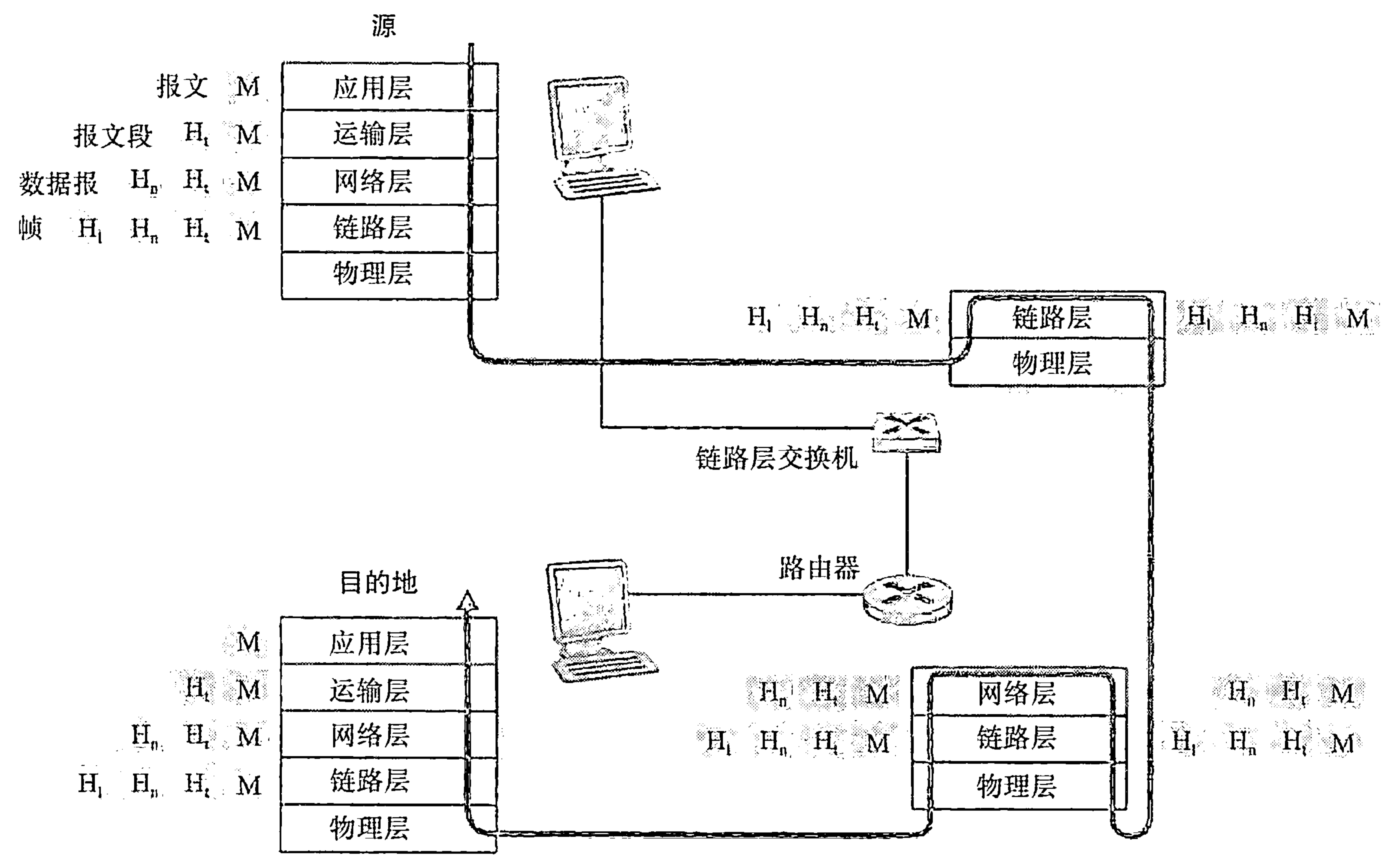


图 1-24 主机、路由器和链路层交换机，每个包含了不同的层，反映了不同的功能

图 1-24 也说明了一个重要概念：封装（encapsulation）。在发送主机端，一个应用层报文（application-layer message）（图 1-24 中的 M）被传送给运输层。在最简单的情况下，运输层收取到报文并附上附加信息（所谓运输层首部信息，图 1-24 中的 H_t），该首部将被接收端的运输层使用。应用层报文和运输层首部信息一道构成了运输层报文段（transport-layer segment）。运输层报文段因此封装了应用层报文。附加的信息也许包括了下列信息：如允许接收端运输层向上向适当的应用程序交付报文的信息；如差错检测位信息，该信息让接收方能够判断报文中的比特是否在途中已被改变。运输层则向网络层传递该报文段，网络层增加了如源和目的端系统地址等网络层首部信息（图 1-24 中的 H_n），产生了网络层数据报（network-layer datagram）。该数据报接下来被传递给链路层，链路层（自然而然地）增加它自己的链路层首部信息并创建链路层帧（link-layer frame）。所以，我们看到在

每一层，一个分组具有两种类型的字段：首部字段和有效载荷字段（payload field）。有效载荷通常是来自上一层的分组。

这里一个有用的类比是经过公共邮政服务在某公司分支办公室之间发送一封备忘录。假定位于一个分支办公室的 Alice 要向位于另一个分支办公室的 Bob 发送一封备忘录。该备忘录类比于应用层报文。Alice 将备忘录放入办公室之间的公函信封中，并在公函信封上方写上了 Bob 的名字和部门编号。该办公室之间的公函信封类比于运输层报文段，即它包括了首部信息（Bob 的名字和部门编码）并封装了应用层报文（备忘录）。当发送方分支办公室的收发室拿到该办公室之间的备忘录，将其放入适合在公共邮政服务发送的信封中，并在邮政信封上写上发送和接收分支办公室的邮政地址。此处，邮政信封类比于数据报，它封装了运输层的报文段（办公室之间信封），该报文段封装了初始报文（备忘录）。邮政服务将该邮政信封交付给接收方分支办公室的收发室。在此处开始了拆封过程。该收发室抽取了办公室间的备忘录并转发给 Bob。最后，Bob 打开封套并拿走了备忘录。

封装的过程能够比前面描述的更为复杂。例如，一个大报文可能被划分为多个运输层的报文段（这些报文段每个可能被划分为多个网络层数据报）。在接收端，则必须从其连续的数据报中重构这样一个报文段。

1.6 面对攻击的网络

对于今天的许多机构（包括大大小小的公司、大学和政府机关）而言，因特网已经成为与其使命密切相关的一部分了。许多个人也依赖因特网进行许多职业、社会和个人活动。但是在所有这一切背后，存在着一个阴暗面，其中的“坏家伙”试图对我们的日常生活中进行破坏，如损坏我们与因特网相连的计算机，侵犯我们的隐私以及使我们依赖的因特网服务无法运行。

网络安全领域主要探讨以下问题：坏家伙如何攻击计算机网络，以及我们（即将成为计算机网络的专家）如何防御以免受他们的攻击，或者更好的是设计能够事先免除这样的攻击的新型体系结构。面对经常发生的各种各样的现有攻击以及新型和更具摧毁性的未来攻击的威胁，网络安全已经成为近年来计算机网络领域的中心主题。本书的特色之一是将网络安全问题放在中心位置。

因为我们在计算机网络和因特网协议方面还没有专业知识，所以这里我们将开始审视某些今天最为流行的与安全性相关的问题。这将刺激我们的胃口，以便我们在后续章节中进行更为充实的讨论。我们在这里以提出问题开始：什么东西会出现问题？计算机网络是怎样易于受到攻击的？今天某些最为流行的攻击类型是什么？

1. 坏家伙能够经因特网将有害程序放入你的计算机中

因为我们要从/向因特网接收/发送数据，所以我们将设备与因特网相连。这包括各种好东西，例如 Web 页面、电子邮件报文、MP3、电话、视频实况、搜索引擎结果等。但是，不幸的是伴随好的东西而来的还有恶意的东西（这些恶意的东西可统称为恶意软件（malware）），它们能够进入并影响我们的设备。一旦恶意软件感染我们的设备，就能够做各种不正当的事情，包括删除我们的文件，安装间谍软件来收集我们的隐私信息，如社会保险号、口令和击键，然后将这些（当然经因特网）发送给坏家伙。我们的受害主机也可能成为数以千计的类似受害设备网络中的一员，它们被统称为僵尸网络（botnet），坏家伙

利用僵尸网络控制并有效地对目标主机展开垃圾邮件分发或分布式拒绝服务攻击（很快将讨论）。

今天的多数恶意软件是自我复制（self-replicating）的：一旦它感染了一台主机，就会从那台主机寻求进入更多的主机。以这种方式，自我复制的恶意软件能够指数式地快速扩散。恶意软件能够以病毒或蠕虫的形式扩散。病毒（virus）是一种需要某种形式的用户交互来感染用户设备的恶意软件。典型的例子是包含恶意可执行代码的电子邮件附件。如果用户接收并打开这样的附件，不经意间就在其设备上运行了该恶意软件。通常，这种电子邮件病毒是自我复制的：例如，一旦执行，该病毒可能向用户地址簿上的每个接收方发送一个具有相同恶意附件的相同报文。蠕虫（worm）是一种无需任何明显用户交互就能进入设备的恶意软件。例如，用户也许运行了一个某攻击者能够发送恶意软件的脆弱网络应用程序。在某些情况下，没有用户的任何干预，该应用程序可能从因特网接收恶意软件并运行它，生成了蠕虫。新近感染设备中的蠕虫则能扫描因特网，搜索其他运行相同易受感染的网络应用程序的主机。当它发现其他易受感染的主机时，便向这些主机发送一个它自身的副本。今天，恶意软件无所不在且造成的损失惨重。当你用这本书学习时，我们鼓励你思考下列问题：计算机网络设计者能够采取什么防御措施，以使与因特网连接的设备免受恶意软件的攻击？

2. 坏家伙能够攻击服务器和网络基础设施

另一种宽泛类型的安全性威胁称为拒绝服务攻击（Denial-of-Service（DoS）attack）。顾名思义，DoS 攻击使得网络、主机或其他基础设施部分不能由合法用户所使用。Web 服务器、电子邮件服务器、DNS 服务器（在第 2 章中讨论）和机构网络都能够成为 DoS 攻击的目标。因特网 DoS 攻击极为常见，每年会出现数以千计的 DoS 攻击 [Moore 2001; Mirkovic 2005]。大多数因特网 DoS 攻击属于下列三种类型之一：

- 弱点攻击。这涉及向一台目标主机上运行的易受攻击的应用程序或操作系统发送制作精细的报文。如果适当顺序的多个分组发送给一个易受攻击的应用程序或操作系统，该服务器可能停止运行，或者更糟糕的是主机可能崩溃。
- 带宽洪泛。攻击者向目标主机发送大量的分组，分组数量之多使得目标的接入链路变得拥塞，使得合法的分组无法到达服务器。
- 连接洪泛。攻击者在目标主机中创建大量的半开或全开 TCP 连接（将在第 3 章中讨论 TCP 连接）。该主机因这些伪造的连接而陷入困境，并停止接受合法的连接。

我们现在更详细地研究这种带宽洪泛攻击。回顾 1.4.2 节中讨论的时延和丢包问题，显然，如果某服务器的接入速率为 R bps，则攻击者将需要以大约 R bps 的速率来产生危害。如果 R 非常大的话，单一攻击源可能无法产生足够大的流量来伤害该服务器。此外，如果从单一源发出所有流量的话，上游路由器就能够检测出该攻击并在该流量靠近服务器前就将其阻挡下来。在图 1-25 中显示的分布式 DoS（Distributed DoS，DDoS）中，攻击者控制多个源并让每个源向目标猛烈发送流量。使用这种方法，为了削弱或损坏服务器，遍及所有受控源的聚合流量速率需要大约 R 的能力。DDoS 攻击充分利用具有数以千计的受害主机的僵尸网络在今天是屡见不鲜的 [Mirkovic 2005]。相比于来自单一主机的 DoS 攻击，DDoS 攻击更加难以检测和防范。

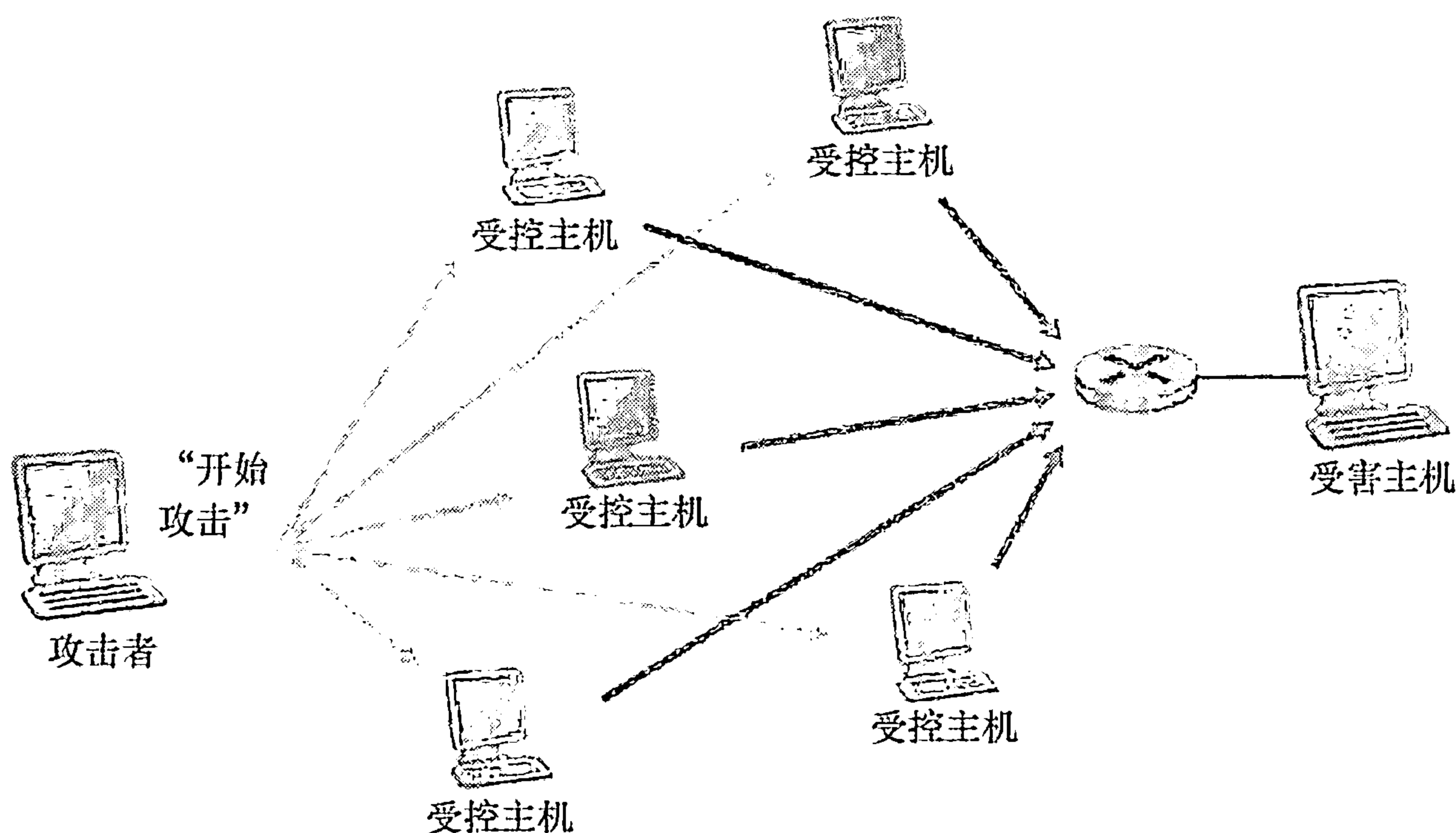


图 1-25 分布式拒绝服务攻击

当学习这本书时，我们鼓励你考虑下列问题：计算机网络设计者能够采取哪些措施防止 DoS 攻击？我们将看到，对于 3 种不同类型的 DoS 攻击需要采用不同的防御方法。

3. 坏家伙能够嗅探分组

今天的许多用户经无线设备接入因特网，如 WiFi 连接的膝上电脑或使用蜂窝因特网连接的手持设备（在第 6 章中讨论）。当无所不在的因特网接入极为便利并使得令人惊奇的新应用程序为移动用户所用的同时，也产生了重大的安全弱点——在无线传输设备的附近放置一台被动的接收机，该接收机就能得到传输的每个分组的副本！这些分组包含了各种敏感信息，包括口令、社会保险号、商业秘密和隐秘的个人信息。记录每个流经的分组副本的被动接收机被称为分组嗅探器（packet sniffer）。

嗅探器也能够部署在有线环境中。在有线的广播环境中，如在许多以太网 LAN 中，分组嗅探器能够获得经该 LAN 发送的所有分组。如在 1.2 节中描述的那样，电缆接入技术也广播分组，因此易于受到嗅探攻击。此外，获得某机构与因特网连接的接入路由器或接入链路访问权的坏家伙能够放置一台嗅探器以产生从该机构出入的每个分组的副本，再对嗅探到的分组进行离线分析，就能得出敏感信息。

分组嗅探软件在各种 Web 站点上可免费得到，这类软件，也有商用的产品。教网络课程的教授们布置的实验作业就涉及写一个分组嗅探器和应用层数据重构程序。与本书相关联的 Wireshark [Wireshark 2012] 实验（参见本章结尾处的 Wireshark 实验介绍）使用的正是这样一种分组嗅探器！

因为分组嗅探器是被动的，也就是说它们不向信道中注入分组，所以难以检测它们的存在。因此，当我们向无线信道发送分组时，我们必须接受这样的可能性，即某些坏家伙可能记录了我们的分组的副本。如你已经猜想的那样，最好的防御嗅探的方法基本上都与密码学有关。我们将在第 8 章研究密码学应用于网络安全的有关内容。

4. 坏家伙能够伪装成你信任的人

生成具有任意源地址、分组内容和目的地址的分组，然后将这个人工制作的分组传输到因特网中极为容易（当你学完这本教科书后，你将很快具有这方面的知识了！），

因特网将忠实地将该分组转发到目的地。想象某接收到这样一个分组的不可信的接收方（比如说一台因特网路由器），将该（虚假的）源地址作为真实的，进而执行某些嵌入在该分组内容中的命令（比如说修改它的转发表）。将具有虚假源地址的分组注入因特网的能力被称为 IP 哄骗（IP spoofing），而它只是一个用户能够冒充另一个用户的许多方式中的一种。

为了解决这个问题，我们需要采用端点鉴别，即一种使我们能够确信一个报文源自我们认为它应当来自的地方的机制。当你继续学习本书各章时，再次建议你思考怎样为网络应用程序和协议做这件事。我们将在第 8 章研究端点鉴别机制。

在本节结束时，值得思考一下因特网是如何从一开始就落入这样一种不安全的境地的。大体上讲，答案是：因特网最初就是基于“一群相互信任的用户连接到一个透明的网络上”这样的模型 [Blumenthal 2001] 进行设计的，在这样的模型中，安全性没有必要。初始的因特网体系结构的许多方面都深刻地反映了这种相互信任的观念。例如，一个用户向任何其他用户发送分组的能力是默认的，而不是一种请求/准予的能力，还有用户身份取自所宣称的表面价值，而不是默认地需要鉴别。

但是今天的因特网无疑并不涉及“相互信任的用户”。无论如何，今天的用户并不是必须相互信任的，他们仍然需要通信，也许希望匿名通信，也许间接地通过第三方通信（例如我们将在第 2 章中学习的 Web 高速缓存，我们将在第 6 章学习的移动性协助代理），也许不信任他们通信时使用的硬件、软件甚至空气。随着我们进一步学习本书，会面临许多安全性相关的挑战：我们应当寻求对嗅探、端点假冒、中间人攻击、DDoS 攻击、恶意软件等的防护办法。我们应当记住：在相互信任的用户之间的通信是一种例外而不是规则。欢迎您到现代计算机网络世界来！

1.7 计算机网络和因特网的历史

1.1 节到 1.6 节概述了计算机网络和因特网的技术。你现在应当有足够的知识来给家人和朋友留下深刻印象了。然而，如果你真的想在下次鸡尾酒会上一鸣惊人，你应当在你的演讲中点缀上一些有关因特网引人入胜的历史轶闻 [Segaller 1998]。

1.7.1 分组交换的发展：1961 ~ 1972

计算机网络和今天因特网领域的开端可以回溯到 20 世纪 60 年代的早期，那时电话网是世界上占统治地位的通信网络。1.3 节讲过，电话网使用电路交换将信息从发送方传输到接收方，这种适当的选择使得语音以一种恒定的速率在发送方和接收方之间传输。随着 20 世纪 60 年代早期计算机的重要性越来越大，以及分时计算机的出现，考虑如何将计算机连接在一起，并使它们能够被地理上分布的用户所共享的问题，也许就成了一件自然的事。这些用户所产生的流量很可能具有“突发性”，即活动的间断性，例如向远程计算机发送一个命令，接着由于在等待应答或在对接收到的响应进行思考而有静止的时间段。

全世界有 3 个研究组首先发明了分组交换，以作为电路交换的一种有效的、健壮的替代技术。这 3 个研究组互不知道其他人的工作 [Leiner 1998]。有关分组交换技术的首次公开发表出自 Leonard Kleinrock [Kleinrock 1961, Kleinrock 1964]，那时他是麻省理工学

院（MIT）的一名研究生。Kleinrock 使用排队论，完美地体现了使用分组交换方法处理突发性流量源的有效性。1964 年，兰德公司的 Paul Baran [Baran 1964] 已经开始研究分组交换的应用，即在军用网络上传输安全语音，同时在美国的国家物理实验室（NPL），Donald Davies 和 Roger Scantlebury 也在研究分组交换技术。

MIT、兰德和 NPL 的工作奠定了今天的因特网的基础。但是因特网也经历了很长的“边构建边示范（let's-build-it-and-demonstrate-it）”的历史，这可追溯到 20 世纪 60 年代早期。J. C. R. Licklider [DEC 1990] 和 Lawrence Roberts 都是 Kleinrock 在 MIT 的同事，他们转而去领导美国高级研究计划署（Advanced Research Projects Agency, ARPA）的计算机科学计划。Roberts 公布了一个号称 ARPAnet [Roberts 1967] 的总体计划，它是第一个分组交换计算机网络，是今天的公共因特网的直接祖先。在 1969 年的劳动节，第一台分组交换机在 Kleinrock 的监管下安装在 UCLA（美国加州大学洛杉矶分校），其他 3 台分组交换机不久后安装在斯坦福研究所（Stanford Research Institute, SRI）、美国加州大学圣巴巴拉分校（UC Santa Barbara）和犹他大学（University of Utah）（参见图 1-26）。羽翼未丰的因特网祖先到 1969 年年底有了 4 个结点。Kleinrock 回忆说，该网络的最先应用是从 UCLA 到 SRI 执行远程注册，但却导致了该系统的崩溃 [Kleinrock 2004]。

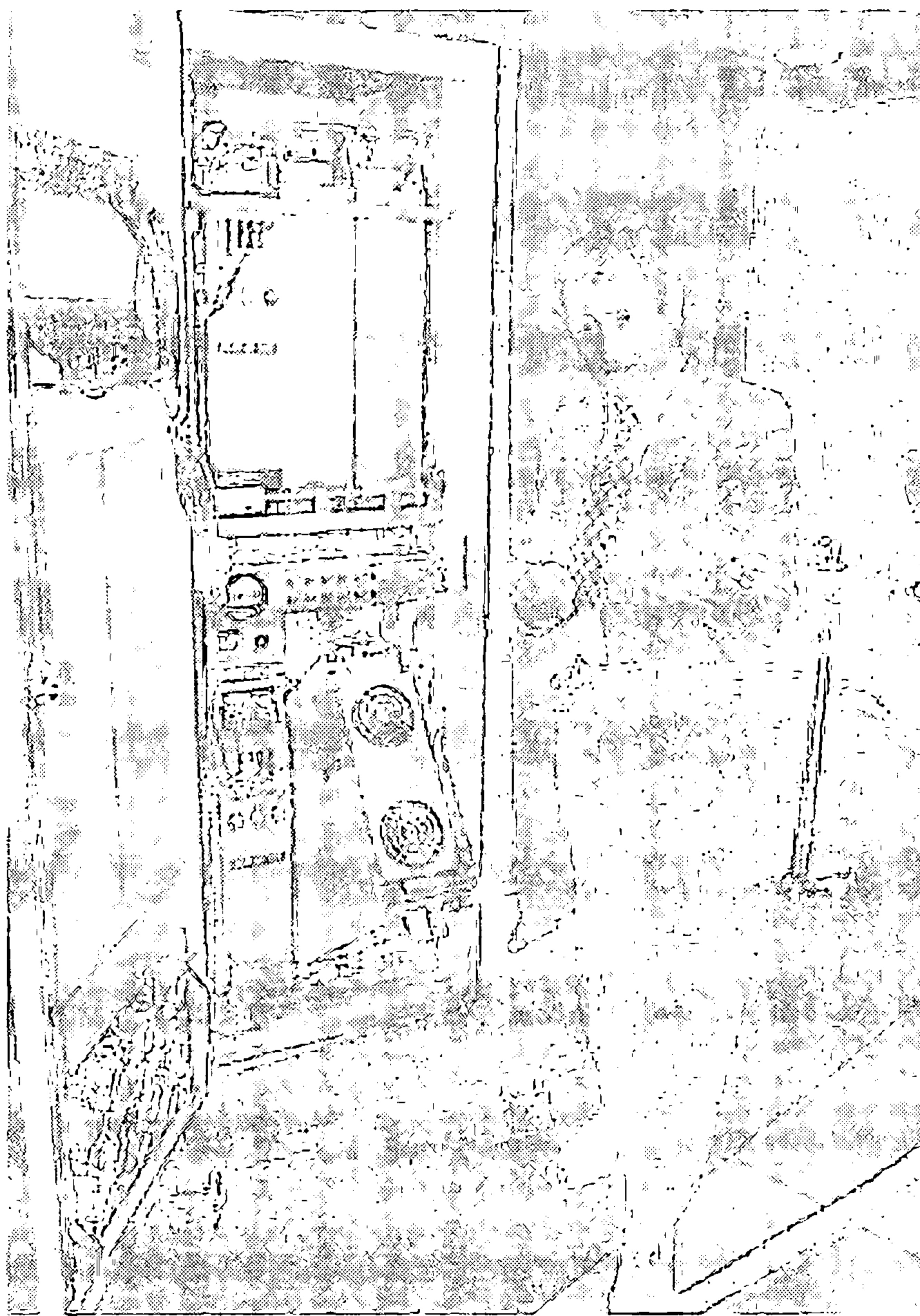


图 1-26 一台早期的分组交换机

到了 1972 年，ARPAnet 已经成长为约 15 个结点，由 Robert Kahn 首次对它进行了公众演示。在 ARPAnet 端系统之间的第一台主机到主机协议称为网络控制协议（NCP），就是此时完成的 [RFC 001]。随着端到端协议的可供使用，这时能够写应用程序了。在

1972 年, Ray Tomlinson 编写了第一个电子邮件程序。

1.7.2 专用网络和网络互联: 1972 ~ 1980

最初的 ARPAnet 是一个单一的、封闭的网络。为了与 ARPAnet 的一台主机通信, 一台主机必须与另一台 ARPAnet IMP 实际相连。从 20 世纪 70 年代早期和中期, 除 ARPAnet 之外的其他分组交换网络问世: ALOHAnet 是一个微波网络, 它将夏威夷岛上的大学 [Abramson 1970], 以及 DARPA 的分组卫星 [RFC 829] 和分组无线电网 [Kahn 1987] 连接到一起; Telenet 是 BBN 的商用分组交换网, 基于 ARPAnet 技术; 由 Louis Pouzin 领衔的 Cyclades 是一个法国的分组交换网 [Think 2012]; 如 Tymnet 和 GE 信息服务网这样的分时网络, 以及 20 世纪 60 年代后期和 70 年代初期的类似网络 [Schwartz 1977]; IBM 的 SNA (1966 ~ 1974), 它与 ARPAnet 并行工作 [Schwartz 1977]。

网络的数目开始增加。时至今日人们看到, 研制将网络连接到一起的体系结构的时机已经成熟。互联网络的前驱性工作 (得到了美国国防部高级研究计划署 (DARPA) 的支持) 由 Vinton Cerf 和 Robert Kahn [Cerf 1974] 完成, 本质上就是创建一个网络的网络; 术语网络互联 (internetting) 就是用来描述该项工作的。

这些体系结构的原则被具体表达在 TCP 协议中。然而, TCP 的早期版本与今天的 TCP 差异很大。TCP 的早期版本与数据可靠的顺序传递相结合, 经过具有转发功能 (今天该功能由 IP 执行) 的端系统的重传 (仍是今天的 TCP 的一部分)。TCP 的早期实验以及认识到对诸如分组语音这样的应用程序——一个不可靠、非流控制的端到端传递服务的重要性, 导致 IP 从 TCP 中分离出来, 并研制了 UDP 协议。我们今天看到的 3 个重要的因特网协议——TCP、UDP 和 IP, 到 20 世纪 70 年代末在概念上已经完成。

除了 DARPA 的因特网相关研究外, 许多其他重要的网络活动也在进行中。在夏威夷, Norman Abramson 正在研制 ALOHAnet, 这是一个基于分组的无线网络, 它使在夏威夷岛上的多个远程站点互相通信。ALOHA 协议 [Abramson 1970] 是第一个多路访问协议, 允许地理上分布的用户共享单一的广播通信媒体 (一个无线电频率)。Metcalfe 和 Boggs 基于 Abramson 的多路访问协议, 研制了基于有线的共享广播网络的以太网协议 [Metcalfe 1976]。令人感兴趣的是, Metcalfe 和 Boggs 的以太网协议是由连接多台 PC、打印机和共享磁盘在一起的需求所激励的 [Perkins 1994]。在 PC 革命和网络爆炸的 25 年之前, Metcalfe 和 Boggs 就奠定了今天 PC LAN 的基础。

1.7.3 网络的激增: 1980 ~ 1990

到了 20 世纪 70 年代末, 大约 200 台主机与 ARPAnet 相连。到了 20 世纪 80 年代, 连到公共因特网的主机数量达到 100 000 台。20 世纪 80 年代是联网主机数量急剧增长的时期。

这种增长导致了几个创建计算机网络将大学连接到一起的显著成果。BITNET 为位于美国东北部的几个大学之间提供了电子邮件和文件传输。建立了 CSNET (计算机科学网), 以将还没有接入 ARPAnet 的大学研究人员连接在一起。1986 年, 建立了 NSFNET, 为 NSF 资助的超级计算中心提供接入。NSFNET 最初具有 56kbps 的主干速率, 到了 20 世纪 80 年代末, 它的主干运行速率是 1.5Mbps, 并成为连接区域网络的基本主干。

在 ARPAnet 界中, 许多今天的因特网体系结构的最终部分逐渐变得清晰起来。1983

年 1 月 1 日见证了 TCP/IP 作为 ARPAnet 的新的标准主机协议的正式部署，替代了 NCP 协议。从 NCP 到 TCP/IP 的迁移 [RFC 801] 是一个标志性事件，所有主机被要求在那天转移到 TCP/IP 上去。在 20 世纪 80 年代后期，TCP 进行了重要扩展，以实现基于主机的拥塞控制 [Jacobson 1988]。还研制出了 DNS（域名系统），用于把人可读的因特网名字（例如 gaia.cs.umass.edu）映射到它的 32 比特 IP 地址 [RFC 1034]。

在 20 世纪 80 年代初期，在 ARPAnet（这绝大多数是美国努力的成果）发展的同时，法国启动了 Minitel 项目，这个雄心勃勃的计划是让数据网络进入每个家庭。在法国政府的支持下，Minitel 系统由公共分组交换网络（基于 X.25 协议集）、Minitel 服务器和具有内置低速调制解调器的廉价终端组成。Minitel 于 1984 年取得了巨大的成功，当时法国政府向每个需要的住户免费分发一个 Minitel 终端。Minitel 站点包括免费站点（如电话目录站点），以及一些专用站点。这些专用站点根据每个用户的使用来收取费用。在 20 世纪 90 年代中期的鼎盛时期，Minitel 提供了 20 000 多种服务，涵盖从家庭银行到特殊研究数据库的广泛范围。Minitel 在大量法国家庭中存在于 10 年后，大多数美国人才听说因特网。

1.7.4 因特网爆炸：20 世纪 90 年代

20 世纪 90 年代出现了许多标志因特网持续革命和很快到来的商业化的事件。作为因特网祖先的 ARPAnet 已不复存在。1991 年，NSFNET 解除了对 NSFNET 用于商业目的的限制。NSFNET 自身于 1995 年退役，这时因特网主干流量则由商业因特网服务提供商负责承载。

然而，20 世纪 90 年代的主要事件是万维网（World Wide Web）应用程序的出现，它将因特网带入世界上数以百万计的家庭和企业中。Web 作为一个平台，也引入和配置了数百个新的应用程序，其中包括搜索（如谷歌和 Bing）、因特网商务（如亚马逊和 eBay）和社交网络（如脸谱），对这些应用程序我们今天已经习以为常了。

Web 是由 Tim Berners-Lee 于 1989 ~ 1991 年期间在 CERN 发明的 [Berners-Lee 1989]，最初的想法源于 20 世纪 40 年代 Vannevar Bush [Bush 1945] 和 20 世纪 60 年代以来 Ted Nelson [Xanadu 2007] 在超文本方面的早期工作。Berners-Lee 和他的同事研制了 HTML、HTTP、Web 服务器和浏览器的初始版本，这是 Web 的 4 个关键部分。到了 1993 年末前后，大约有 200 台 Web 服务器在运行，而这些只是正在出现的 Web 服务器的冰山一角。就在这个时候，几个研究人员研制了具有 GUI 接口的 Web 浏览器，其中的 Marc Andreessen 和 Jim Clark 一起创办了 Mosaic Communications 公司，该公司就是后来的 Netscape 通信公司 [Cusmano 1998; Quittner 1998]。到了 1995 年，大学生们每天都在使用 Netscape 浏览器在 Web 上冲浪。大约在这段时间，大大小小的公司都开始运行 Web 服务器，并在 Web 上处理商务。1996 年，微软公司开始开发浏览器，这导致了 Netscape 和微软之间的浏览器之战，并以微软公司在几年后获胜而告终 [Cusumano 1998]。

20 世纪 90 年代的后 5 年是因特网飞速增长和变革的时期，伴随着主流公司和数以千计的后起之秀创造因特网产品和服务。到了 2000 年末，因特网已经支持数百流行的应用程序，包括以下 4 种备受欢迎的应用程序：

- 电子邮件，包括附件和 Web 可访问的电子邮件。
- Web，包括 Web 浏览和因特网商务。

- 即时讯息 (instant messaging), 具有联系人列表。
- MP3 的对等 (peer-to-peer) 文件共享, 由 Napster 所领衔。

值得一提的是, 前两个应用程序出自专业研究机构, 而后两个却由一些年轻创业者所发明。

1995 ~ 2001 年, 这段时间也是因特网在金融市场上急转突变的时期。在成为有利可图的公司之前, 数以百计的新兴因特网公司靠首次公开募股 (IPO) 并在股票市场上交易起家。许多公司身价数十亿美元, 却没有任何主要的收入渠道。因特网的股票在 2000 ~ 2001 年崩盘, 导致许多创业公司倒闭。不过, 也有许多公司成为因特网世界的大赢家, 包括微软、思科、雅虎、e-Bay、谷歌和亚马逊。

1.7.5 最新发展

计算机网络中的变革继续以急促的步伐前进。所有的前沿研究正在取得进展, 包括部署更快的路由器和在接入网和网络主干中提供更高的传输速率。但下列进展值得特别关注:

- 自 2000 年开始, 我们见证了家庭宽带因特网接入的积极发展——不仅有电缆调制解调器和 DSL, 而且有光纤到户, 这些在 1.2 节中讨论过。这种高速因特网为丰富的视频应用创造了条件, 包括用户生成的视频的分发 (例如 YouTube), 电影和电视节目流的点播 (例如 Netflix) 和多人视频会议 (例如 Skype)。
- 高速 (54Mbps 及更高) 公共 WiFi 网络和经过 3G 和 4G 蜂窝电话网的中速 (高达几 Mbps) 因特网接入越来越普及, 不仅使在运动中保持持续连接成为可能, 也产生了新型特定位置服务。2011 年, 与因特网连接的无线设备的数量超过了有线设备的数量。高速无线接入为手持计算机 (iPhone、安卓手机、iPad 等) 的迅速出现提供了舞台, 这些手持计算机具有对因特网持续不断和无拘束接入的优点。
- 诸如脸谱和推特 (Twitter) 这样的在线社交网络已经在因特网之上构建了巨大的人际网络。许多因特网用户今天主要“活在”脸谱中。通过他们的 API, 在线社交网络为新的联网应用和分布式游戏创建了平台。
- 如在 1.3.3 节中所讨论的, 在线服务提供商如谷歌和微软已经部署了自己的广泛的专用网络。该专用网络不仅将它们分布在全球的数据中心连接在一起, 而且通过直接与较低层 ISP 对等连接, 能够尽可能绕过因特网。因此, 谷歌几乎可以立即提供搜索结果和电子邮件访问, 仿佛它们的数据中心运行在自己的计算机之中一样。
- 许多因特网商务公司在“云” (如亚马逊的 EC2、谷歌的应用引擎、微软的 Azure) 中运行它们的应用。许多公司和大学也已经将它们的因特网应用 (如电子邮件和 Web 集合) 迁移到云中。云公司不仅可以为应用提供可扩展的计算和存储环境, 也可为应用提供对其高性能专用网络的隐含访问。

1.8 小结

在本章中, 我们涉及了大量的材料! 我们已经看到构成特别的因特网和普通的计算机网络的各种硬件和软件。我们从网络的边缘开始, 观察端系统和应用程序, 以及运行在端系统上为应用程序提供的运输服务。接着我们也观察了通常能够在接入网中找到的链路层技术和物理媒体。然后我们进入网络核心更深入地钻研网络, 看到分组交换和电路交换是

通过电信网络传输数据的两种基本方法，并且探讨了每种方法的长处和短处。我们也研究了全球性因特网的结构，知道了因特网是网络的网络。我们看到了因特网的由较高层和较低层 ISP 组成的等级结构，允许该网络扩展为包括数以千计的网络。

在这个概述性一章的第二部分，我们研究了计算机网络领域的几个重要主题。我们首先研究了分组交换网中的时延、吞吐量和丢包的原因。我们研究得到传输、传播和排队时延以及用于吞吐量的简单定量模型；我们将在整本书的课后习题中多处使用这些时延模型。接下来，我们研究了协议分层和服务模型、联网中的关键体系结构原则，我们将在本书多处引用它们。我们还概述了在今天的因特网中某些更为盛行的安全攻击。我们用计算机网络的简要历史结束我们对网络的概述。第 1 章本身就构成了计算机网络的小型课程。

因此，第 1 章中的确涉及了大量的背景知识！如果你有些不知所云，请不要着急。在后继几章中我们将重新回顾这些概念，更为详细地研究它们（那是承诺，而不是威胁！）。此时，我们希望你完成本章内容的学习时，对构建网络的众多元素的直觉越来越敏锐，对网络词汇越来越精通（不妨经常回过头来查阅本章），对更加深入地学习网络的愿望越来越强烈。这些也是在本书的其余部分我们将面临的任务。

本书的路线图

在开始任何旅行之前，你总要先察看路线图，以便更为熟悉前面的主要道路和交界处。对于我们即将开启的这段“旅行”而言，其最终目的地是深入理解计算机网络“是什么、怎么样和为什么”等内容。我们的路线图是本书各章的顺序：

- 第 1 章 计算机网络和因特网
- 第 2 章 应用层
- 第 3 章 运输层
- 第 4 章 网络层
- 第 5 章 链路层：链路、接入网络和局域网
- 第 6 章 无线网络和移动网络
- 第 7 章 多媒体网络
- 第 8 章 计算机网络中的安全
- 第 9 章 网络管理

从第 2 章到第 5 章是本书的 4 个核心章。应当注意的是，这些章都围绕 5 层因特网协议栈上面的 4 层而组织，其中一章对应一层。进一步要注意的是，我们的旅行将从因特网协议栈的顶部，即应用层开始，然后向下面各层进行学习。这种自顶向下旅行背后的基本原理是，一旦我们理解这些应用程序，就能够理解支持这些应用程序所需的网络服务。然后能够依次研究由网络体系结构可能实现的服务的各种方式。较早地涉及应用程序，也能够对学习本课程其余部分提供动力。

第 6 章到第 9 章关注现代计算机网络中的 4 个极为重要的（并且在某种程度上是独立的）主题。在第 6 章中，我们研究了无线网络和移动网络，包括无线 LAN（其中有 WiFi 和蓝牙）、蜂窝电话网（包括 GSM、3G 和 4G）和（在 IP 网络和 GSM 网络中的）移动性。在第 7 章中，我们研究了音频和视频应用，例如因特网电话、视频会议和流式存储媒体。此外，还讨论如何设计分组交换网络以对音频和视频应用程序提供一致的服务质量。在第

8 章中，我们首先学习加密和网络安全的基础知识，然后研究基础理论如何应用于因特网环境的不同情况。最后一章（第 9 章）研究网络管理中的关键问题以及网络管理中使用的因特网协议。

课后习题和问题



复习题

1.1 节

R1. “主机”和“端系统”之间有什么不同？列举几种不同类型的端系统。Web 服务器是一种端系统吗？

R2. “协议”一词常被用于描述外交关系。维基百科是怎样描述外交协议的？

R3. 标准对于协议为什么重要？

1.2 节

R4. 列出 6 种接入技术。将它们分类为住宅接入、公司接入或广域无线接入。

R5. HFC 带宽是专用的，还是用户间共享的？在下行 HFC 信道中，有可能发生碰撞吗？为什么？

R6. 列出你所在城市中的可供使用的住宅接入技术。对于每种类型的接入方式，给出所宣称的下行速率、上行速率和每月的价格。

R7. 以太 LAN 的传输速率是多少？

R8. 能够运行以太网的一些物理媒体是什么？

R9. 拨号调制解调器、HFC、DSL 和 FTTH 都用于住宅接入。对于这些技术中的每一种，给出传输速率的范围，并讨论有关带宽是共享的还是专用的。

R10. 描述今天最为流行的无线因特网接入技术。对它们进行比较和对照。

1.3 节

R11. 假定在发送主机和接收主机间只有一台分组交换机。发送主机和交换机间以及交换机和接收主机间的传输速率分别是 R_1 和 R_2 。假设该交换机使用存储转发分组交换方式，发送一个长度为 L 的分组的端到端总时延是什么？（忽略排队时延、传播时延和处理时延。）

R12. 与分组交换网络相比，电路交换网络有哪些优点？在电路交换网络中，TDM 比 FDM 有哪些优点？

R13. 假定用户共享一条 2Mbps 链路。同时假定当每个用户传输时连续以 1Mbps 传输，但每个用户仅传输 20% 的时间。

a. 当使用电路交换时，能够支持多少用户？

b. 作为该题的遗留问题，假定使用分组交换。为什么如果两个或更少的用户同时传输的话，在链路前面基本上没有排队时延？为什么如果 3 个用户同时传输的话，将有排队时延？

c. 求出某指定用户正在传输的概率。

d. 假定现在有 3 个用户。求出在任何给定的时间，所有 3 个用户在同时传输的概率。求出队列增长的时间比率。

R14. 为什么在等级结构相同级别的两个 ISP 通常互相对等？某 IXP 是如何挣钱的？

R15. 某些内容提供商构建了自己的网络。描述谷歌的网络。内容提供商构建这些网络的动机是什么？

1.4 节

R16. 考虑从某源主机跨越一条固定路由向某目的主机发送一分组。列出端到端时延中的时延组成成分。这些时延中的哪些是固定的，哪些是变化的？

R17. 访问在配套 Web 网站上有关传输时延与传播时延的 Java 小程序。在可用速率、传播时延和可用的分组长度之中找出一种组合，使得该分组的第一个比特到达接收方之前发送方结束了传输。找出另一种组合，使得发送方完成传输之前，该分组的第一个比特到达了接收方。

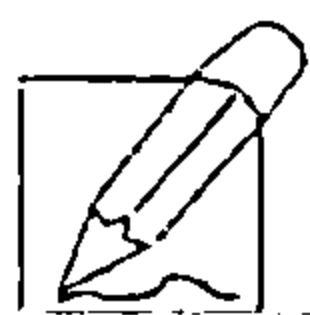
- R18. 一个长度为 1000 字节的分组经距离为 2500km 的链路传播，传播速率为 2.5×10^8 m/s 并且传输速率为 2Mbps，它需要用多长时间？更为一般地，一个长度为 L 的分组经距离为 d 的链路传播，传播速率为 s 并且传输速率为 R bps，它需要用多长时间？该时延与传输速率相关吗？
- R19. 假定主机 A 要向主机 B 发送一个大文件。从主机 A 到主机 B 的路径上有 3 段链路，其速率分别为 $R_1 = 500$ kbps, $R_2 = 2$ Mbps, $R_3 = 1$ Mbps。
- 假定该网络中没有其他流量，该文件传送的吞吐量是多少？
 - 假定该文件为 4MB。传输该文件到主机 B 大致需要多长时间？
 - 重复 (a) 和 (b)，只是这时 R_2 减小到 100 kbps。
- R20. 假定端系统 A 要向端系统 B 发送一个大文件。在一个非常高的层次上，描述端系统怎样从该文件生成分组。当这些分组之一到达某分组交换机时，该交换机使用分组中的什么信息来决定将该分组转发到哪一条链路上？因特网中的分组交换为什么可以与驱车从一个城市到另一个城市并沿途询问方向相类比？
- R21. 访问配套 Web 站点的排队和丢包 Java 小程序。最大发送速率和最小的传输速率是多少？对于这些速率，流量强度是多少？用这些速率运行该 Java 小程序并确定出现丢包要花费多长时间？然后第二次重复该实验，再次确定出现丢包花费多长时间。这些值有什么不同？为什么会有这种现象？

1.5 节

- R22. 列出一个层次能够执行的 5 个任务。这些任务中的一个（或两个）可能由两个（或更多）层次执行吗？
- R23. 因特网协议栈中的 5 个层次有哪些？在这些层次中，每层的主要任务是什么？
- R24. 什么是应用层报文？什么是运输层报文段？什么是网络层数据报？什么是链路层帧？
- R25. 路由器处理因特网协议栈中的哪些层次？链路层交换机处理的是哪些层次？主机处理的是哪些层次？

1.6 节

- R26. 病毒和蠕虫之间有什么不同？
- R27. 描述如何产生一个僵尸网络，以及僵尸网络是怎样被用于 DDoS 攻击的。
- R28. 假定 Alice 和 Bob 经计算机网络互相发送分组。假定 Trudy 将自己安置在网络中，使得她能够俘获由 Alice 发送的所有分组，并发送她希望给 Bob 的东西；她也能够俘获由 Bob 发送的所有分组，并发送她希望给 Alice 的东西。列出在这种情况下 Trudy 能够做的某些恶意的事情。



习题

- P1. 设计并描述在自动柜员机和银行的中央计算机之间使用的一种应用层协议。你的协议应当允许验证用户卡和口令、查询账目结算（这些都在中央计算机系统中进行维护）、支取账目（即向用户支付钱）。你的协议实体应当能够处理取钱时账目中钱不够的常见问题。通过列出自动柜员机和银行中央计算机在报文传输和接收过程中交换的报文和采取的动作来定义你的协议。使用类似于图 1-2 所示的图，拟定在简单无差错取钱情况下该协议的操作。明确地阐述在该协议中关于底层端到端运输服务所作的假设。
- P2. 式 (1-1) 给出了经传输速率为 R 的 N 段链路发送长度 L 的一个分组的端到端时延。对于经过 N 段链路连续地发送 P 个这样的分组，一般化地表示出这个公式。
- P3. 考虑一个应用程序以稳定的速率传输数据（例如，发送方每 k 个时间单元产生一个 N 比特的数据单元，其中 k 较小且固定）。另外，当这个应用程序启动时，它将连续运行相当长的一段时间。回答下列问题，简要论证你的回答：
- 是分组交换网还是电路交换网更为适合这种应用？为什么？
 - 假定使用了分组交换网，并且该网中的所有流量都来自如上所述的这种应用程序。此外，假定该应用程序数据传输速率的总和小于每条链路的各自容量。需要某种形式的拥塞控制吗？为什么？

- P4. 考虑在图 1-13 中的电路交换网。回想在每条链路上有 4 条链路，以顺时针方向标记四台交换机 A、B、C 和 D。
- 在该网络中，任何时候能够进行同时连接的最大数量是多少？
 - 假定所有连接位于交换机 A 和 C 之间。能够进行同时连接的最大数量是多少？
 - 假定我们要在交换机 A 和 C 之间建立 4 条连接，在交换机 B 和 D 之间建立另外 4 条连接。我们能够让这些呼叫通过这 4 条链路建立路由以容纳所有 8 条连接吗？
- P5. 回顾在 1.4 节中的车队的类比。假定传播速度还是 100km/h。
- 假定车队旅行 150km：在一个收费站前面开始，通过第二个收费站，并且在第三个收费站后面结束。其端到端时延是多少？
 - 重复 (a)，现在假定车队中有 8 辆汽车而不是 10 辆。
- P6. 这个习题开始探讨传播时延和传输时延，这是数据网络中的两个重要概念。考虑两台主机 A 和 B 由一条速率为 R bps 的链路相连。假定这两台主机相隔 m 米，沿该链路的传播速率为 s m/s。主机 A 向主机 B 发送长度 L 比特的分组。
- 用 m 和 s 来表示传播时延 d_{prop} 。
 - 用 L 和 R 来确定该分组的传输时间 d_{trans} 。
 - 忽略处理和排队时延，得出端到端时延的表达式。
 - 假定主机 A 在时刻 $t = 0$ 开始传输该分组。在时刻 $t = d_{\text{trans}}$ ，该分组的最后一个比特在什么地方？
 - 假定 d_{prop} 大于 d_{trans} 。在时刻 $t = d_{\text{trans}}$ ，该分组的第一个比特在何处？
 - 假定 d_{prop} 小于 d_{trans} 。在时刻 $t = d_{\text{trans}}$ ，该分组的第一个比特在何处？
 - 假定 $s = 2.5 \times 10^8$ ， $L = 120$ 比特， $R = 56$ kbps。求出使 d_{prop} 等于 d_{trans} 的距离 m 。
- P7. 在这个习题中，我们考虑从主机 A 向主机 B 通过分组交换网发送语音 (VoIP)。主机 A 将模拟语音转换为传输中的 64kbps 数字比特流。然后主机 A 将这些比特分为 56 字节的分组。A 和 B 之间有一条链路：它的传输速率是 2Mbps，传播时延是 10ms。一旦 A 收集了一个分组，就将它向主机 B 发送。一旦主机 B 接收到一个完整的分组，它将该分组的比特转换成模拟信号。从比特产生（从位于主机 A 的初始模拟信号起）的时刻起，到该比特被解码（在主机 B 上作为模拟信号的一部分），花了多少时间？
- P8. 假定用户共享一条 3Mbps 的链路。又设每个用户传输时要求 150kbps，但是每个用户仅有 10% 的时间传输。（参见 1.3 节中关于“分组交换与电路交换的对比”的讨论。）
- 当使用电路交换时，能够支持多少用户？
 - 对于本习题的后续小题，假定使用分组交换。求出给定用户正在传输的概率。
 - 假定有 120 个用户。求出在任何给定时刻，实际有 n 个用户在同时传输的概率。（提示：使用二项式分布。）
 - 求出有 21 个或更多用户同时传输的概率。
- P9. 考虑在 1.3 节“分组交换与电路交换的对比”的讨论中，给出了一个具有一条 1Mbps 链路的例子。用户在忙时以 100kbps 速率产生数据，但忙时仅以 $p = 0.1$ 的概率产生数据。假定用 1Gbps 链路替代 1Mbps 的链路。
- 当采用电路交换技术时，能被同时支持的最大用户数量 N 是多少？
 - 现在考虑分组交换和有 M 个用户的情况。给出多于 N 用户发送数据的概率公式（用 p 、 M 、 N 表示）。
- P10. 考虑一个长度为 L 的分组从端系统 A 开始，经 3 段链路传送到目的端系统。令 d_i 、 s_i 和 R_i 表示链路 i 的长度、传播速度和传输速率 ($i = 1, 2, 3$)。该分组交换机对每个分组的时延为 d_{proc} 。假定没有排队时延，根据 d_i 、 s_i 、 R_i ($i = 1, 2, 3$) 和 L ，该分组总的端到端时延是什么？现在假定该分组是 1500 字节，在所有 3 条链路上传播时延是 2.5×10^8 m/s，所有 3 条链路的传输速率是 2Mbps，分组交换机的处理时延是 3ms，第一段链路的长度是 5000km，第二段链路的长度是 4000km，并且最后一段链路的长度是 1000km。对于这些值，该端到端时延为多少？

- P11. 在上述习题中, 假定 $R_1 = R_2 = R_3 = R$ 且 $d_{\text{proc}} = 0$ 。进一步假定该分组交换机不存储转发分组, 而是在等待分组到达前立即传输它收到的每个比特。这时端到端时延为多少?
- P12. 一台分组交换机接收一个分组并决定该分组应当转发的出链路。当某分组到达时, 另一个分组正在该出链路上被发送到一半, 还有 4 个其他分组正等待传输。这些分组以到达的次序传输。假定所有分组是 1500 字节并且链路速率是 2Mbps。该分组的排队时延是多少? 在更一般的情况下, 当所有分组的长度是 L , 传输速率是 R , 当前正在传输的分组已经传输了 x 比特, 并且已经在队列中有 n 个分组, 其排队时延是多少?
- P13. a. 假定有 N 个分组同时到达一条当前没有分组传输或排队的链路。每个分组长为 L , 链路传输速率为 R 。对 N 个分组而言, 其平均排队时延是多少?
b. 现在假定每隔 LN/R 秒有 N 个分组同时到达链路。一个分组的平均排队时延是多少?
- P14. 考虑路由器缓存中的排队时延。令 I 表示流量强度; 即 $I = La/R$ 。假定排队时延的形式为 $IL/R(1-I)$, 其中 $I < 1$ 。
a. 写出总时延公式, 即排队时延加上传输时延。
b. 以 L/R 为函数画出总时延的图。
- P15. 令 a 表示在一条链路上分组的到达率 (以分组/秒计), 令 μ 表示一条链路上分组的传输率 (以分组/秒计)。基于上述习题中推导出的总时延公式 (即排队时延加传输时延), 推导出以 a 和 μ 表示的总时延公式。
- P16. 考虑一台路由器缓存前面的一条出链路。在这个习题中, 将使用李特尔 (Little) 公式, 这是排队论中的一个著名公式。令 N 表示在缓存中的分组加上被传输的分组的平均数。令 a 表示到达链路的分组速率。令 d 表示一个分组历经的平均总时延 (即排队时延加传输时延)。李特尔公式是 $N = a \times d$ 。假定该缓存平均包含 10 个分组, 并且平均分组排队时延是 10ms。该链路的传输速率是 100 分组/秒。使用李特尔公式, 在没有丢包的情况下, 平均分组到达率是多少?
- P17. a. 对于不同的处理速率、传输速率和传播时延, 给出 1.4.3 节中式 (1-2) 的一般表达式。
b. 重复 (a), 不过此时假定在每个结点有平均排队时延 d_{queue} 。
- P18. 在一天的 3 个不同的小时内, 在同一个大陆上的源和目的地之间执行 Traceroute。
a. 在这 3 个小时的每个小时中, 求出往返时延的均值和方差。
b. 在这 3 个小时的每个小时中, 求出路径上的路由器数量。在这些时段中, 该路径发生变化了吗?
c. 试图根据源到目的地 Traceroute 分组通过的情况, 辨明 ISP 网络的数量。具有类似名字和/或类似的 IP 地址的路由器应当被认为是同一个 ISP 的一部分。在你的实验中, 在相邻的 ISP 间的对等接口处出现最大的时延了吗?
d. 对位于不同大陆上的源和目的地重复上述内容。比较大陆内部和大陆之间的这些结果。
- P19. a. 访问站点 www.traceroute.org, 并从法国两个不同的城市向位于美国的相同的目的主机执行 Traceroute。在这两个 Traceroute 中, 有多少条链路是相同的? 大西洋沿岸国家的链路相同吗?
b. 重复 (a), 但此时选择位于法国的一个城市和位于德国的另一个城市。
c. 在美国挑选一个城市, 然后向位于中国的两个不同城市的主机执行 Traceroute。在这两次 Traceroute 中有多少链路是相同的? 在到达中国前这两个 Traceroute 分开了吗?
- P20. 考虑对应于图 1-20b 吞吐量的例子。现在假定有 M 对客户-服务器而不是 10 对。用 R_s 、 R_c 和 R 分别表示服务器链路、客户链路和网络链路的速率。假设所有的其他链路都有充足容量, 并且除了由这 M 对客户-服务器产生的流量外, 网络中没有其他流量。推导出由 R_s 、 R_c 、 R 和 M 表示的通用吞吐量表达式。
- P21. 考虑图 1-19b。现在假定在服务器和客户之间有 M 条路径。任两条路径都不共享任何链路。路径 k ($k = 1, \dots, M$) 是由传输速率为 R_1^k 、 R_2^k 、 $\dots$ 、 R_N^k 的 N 条链路组成。如果服务器仅能够使用一条路径向客户发送数据, 则该服务器能够取得的最大吞吐量是多少? 如果该服务器能够使用所有 M 条路径发送数据, 则该服务器能够取得的最大吞吐量是多少?

- P22. 考虑图 1-19b。假定服务器与客户之间的每条链路的丢包概率为 p ，且这些链路的丢包率是独立的。一个（由服务器发送的）分组成功地被接收方收到的概率是多少？如果在从服务器到客户的路径上分组丢失了，则服务器将重传该分组。平均来说，为了使客户成功地接收该分组，服务器将要重传该分组多少次？
- P23. 考虑图 1-19a。假定我们知道沿着从服务器到客户的路径的瓶颈链路是速率为 R_s bps 的第一段链路。假定我们从服务器向客户发送紧接着的一对分组，且沿这条路径没有其他流量。假定每个分组的长度为 L 比特，两条链路具有相同的传播时延 d_{prop} 。
- 在目的地，分组的到达间隔时间有多大？也就是说，从第一个分组的最后一个比特到达第二个分组最后一个比特到达所经过的时间有多长？
 - 现在假定第二段链路是瓶颈链路（即 $R_c < R_s$ ）。第二个分组在第二段链路输入队列中排队是可能的吗？请解释原因。现在假定服务器在发送第一个分组 T 秒之后再发送第二个分组。为确保在第二段链路之前没有排队， T 必须要有多长？试解释原因。
- P24. 假设你希望从波士顿到洛杉矶紧急传送 40×10^{12} 字节数据。你有一条 100Mbps 专用链路可用于传输数据。你是愿意通过这条链路传输数据，还是愿意使用 FedEx 一夜快递？解释你的理由。
- P25. 假定两台主机 A 和 B 相隔 20 000km，由一条直接的 $R = 2$ Mbps 的链路相连。假定跨越该链路的传播速率是 2.5×10^8 m/s。
- 计算带宽-时延积 $R \cdot d_{\text{prop}}$ 。
 - 考虑从主机 A 到主机 B 发送一个 800 000 比特的文件。假定该文件作为一个大的报文连续发送。在任何给定的时间，在链路上具有的比特数量最大值是多少？
 - 给出带宽-时延积的一种解释。
 - 在该链路上一个比特的宽度（以米计）是多少？它比一个足球场更长吗？
 - 根据传播速率 s 、带宽 R 和链路 m 的长度，推导出一个比特宽度的一般表示式。
- P26. 对于习题 P25，假定我们能够修改 R 。对什么样的 R 值，一个比特的宽度能与该链路的长度一样长？
- P27. 考虑习题 P25，但现在链路的速率是 $R = 1$ Gbps。
- 计算带宽-时延积 $R \cdot d_{\text{prop}}$ 。
 - 考虑从主机 A 到主机 B 发送一个 800 000 比特的文件。假定该文件作为一个大的报文连续发送。在任何给定的时间，在链路上具有的比特数量最大值是多少？
 - 在该链路上一个比特的宽度（以米计）是多少？
- P28. 再次考虑习题 P25。
- 假定连续发送，发送该文件需要多长时间？
 - 假定现在该文件被划分为 20 个分组，每个分组包含 40 000 比特。假定每个分组被接收方确认，确认分组的传输时间可忽略不计。最后，假定前一个分组被确认后，发送方才能发送分组。发送该文件需要多长时间？
 - 比较（a）和（b）的结果。
- P29. 假定在同步卫星和它的地球基站之间有一条 10Mbps 的微波链路。每分钟该卫星拍摄一幅数字照片，并将它发送到基站。假定传播速率是 2.4×10^8 m/s。
- 该链路的传播时延是多少？
 - 带宽-时延积 $R \cdot d_{\text{prop}}$ 是多少？
 - 若 x 表示该照片的大小。对于这条微波链路，能够连续传输的 x 最小值是多少？
- P30. 考虑 1.5 节中我们在分层讨论中对航线旅行的类比，随着协议数据单元向协议栈底层流动，首部在增加。随着旅客和行李移动到航线协议栈底部，有与上述首部信息等价的概念吗？
- P31. 在包括因特网的现代分组交换网中，源主机将长应用层报文（如一个图像或音乐文件）分段为较小的分组并向网络发送。接收方则将这些分组重新装配为初始报文。我们称这个过程为报文分段。图 1-27 显示了一个报文在报文不分段或报文分段情况下的端到端传输。考虑一个长度为 8×10^6 比

特的报文，它在图 1-27 中从源发送到目的地。假定在该图中的每段链路是 2Mbps。忽略传播、排队和处理时延。

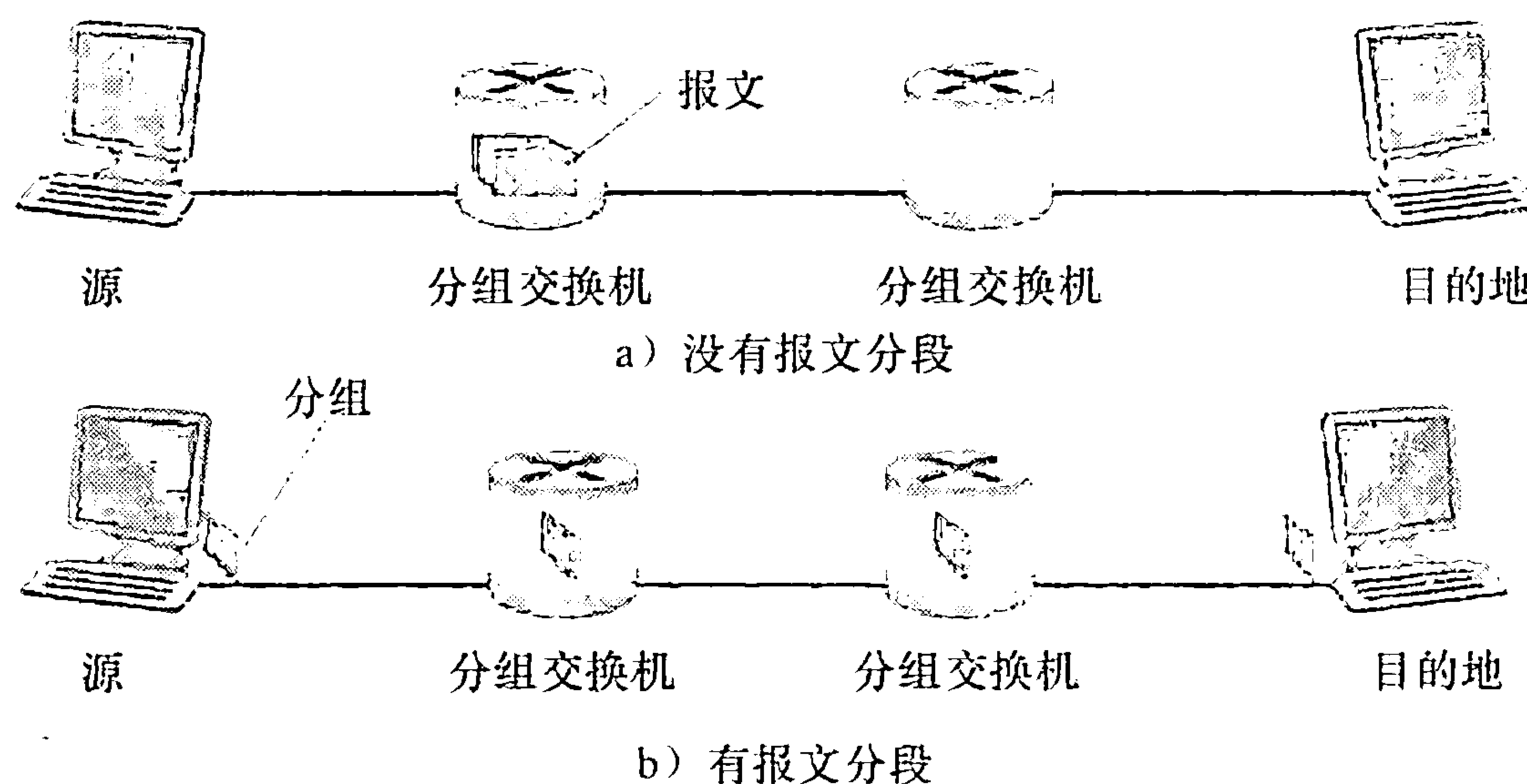


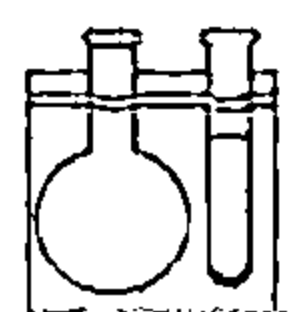
图 1-27 端到端报文传输

- 考虑从源到目的地发送该报文且没有报文分段。从源主机到第一台分组交换机移动报文需要多长时间？记住，每台交换机均使用存储转发分组交换，从源主机移动该报文到目的主机需要多长时间？
- 现在假定该报文被分段为 800 个分组，每个分组 10 000 比特长。从源主机移动第一个分组到第一台交换机需要多长时间？从第一台交换机发送第一个分组到第二台交换机，从源主机发送第二个分组到第一台交换机各需要多长时间？什么时候第二个分组能被第一台交换机全部收到？
- 当进行报文分段时，从源主机向目的主机移动该文件需要多长时间？将该结果与 (a) 的答案进行比较并解释之。
- 除了减小时延外，使用报文分段还有什么原因？
- 讨论报文分段的缺点。

P32. 用本书的 Web 网站上的报文分段小 Java 小程序进行实验。该程序中的时延与前一个习题中的时延相当吗？链路传播时延是怎样影响分组交换（有报文分段）和报文交换的端到端总时延的？

P33. 考虑从主机 A 到主机 B 发送一个 F 比特的大文件。A 和 B 之间有两段链路（和两台交换机），并且该链路不拥塞（即没有排队时延）。主机 A 将该文件分为每个为 S 比特的报文段，并为每个报文段增加一个 80 比特的首部，形成 $L = 80 + S$ 比特的分组。每条链路的传输速率为 R bps。求出从 A 到 B 移动该文件时延最小的值 S 。忽略传播时延。

P34. Skype 提供了一种服务，使你能用 PC 向普通电话打电话。这意味着语音呼叫必须通过因特网和电话网。讨论这是如何做到的。



Wireshark 实验

“不闻不若闻之，闻之不若见之，见之不若知之，知之不若行之。”

——中国谚语

一个人通常能够通过以下方法加深对网络协议的理解：观察它们的动作和经常摆弄它们，即观察两个协议实体之间交换的报文序列，钻研协议运行的细节，使协议执行某些动作，观察这些动作及其后果。这能够在仿真环境下或在如因特网这样的真实网络环境下完成。在本书配套 Web 站点上的 Java 小程序采用的是第一种方法。在 Wireshark 实验中，我们将采用后一种方法。你可以在家中或实验室中使用桌面计算机在各种情况下运行网络应用程序。在你的计算机上观察网络协议，它是如何与在因特网别处执行的协议实体交互和交换报文的。因此，你与你的计算机将是这些真实实验的有机组成部分。你将通过动手来观察和学习。

用来观察执行协议实体之间交换的报文的基本工具称为分组嗅探器（packet sniffer）。顾名思义，一

个分组嗅探器被动地拷贝（嗅探）由你的计算机发送和接收的报文；它也能显示出这些被俘获报文的各个协议字段的内容。图 1-28 中显示了 Wireshark 分组嗅探器的屏幕快照。Wireshark 是一个运行在 Windows、Linux/Unix 和 Mac 计算机上的免费分组嗅探器。贯穿全书，你将发现 Wireshark 实验能让你探索在该章中学习的一些协议。在这第一个 Wireshark 实验中，你将获得并安装一个 Wireshark 的副本，访问一个 Web 站点，俘获并检查在你的 Web 浏览器和 Web 服务器之间交换的协议报文。

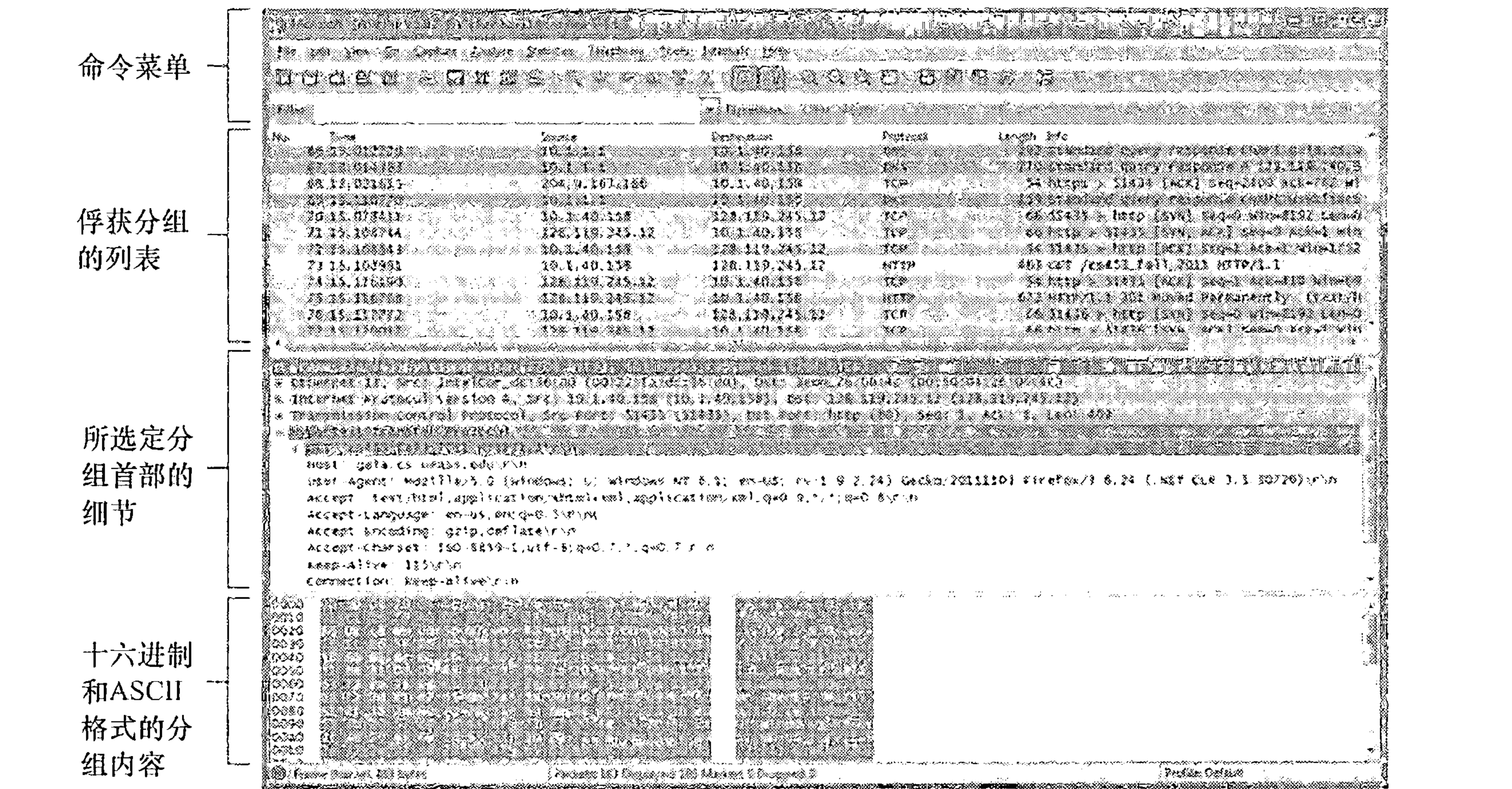
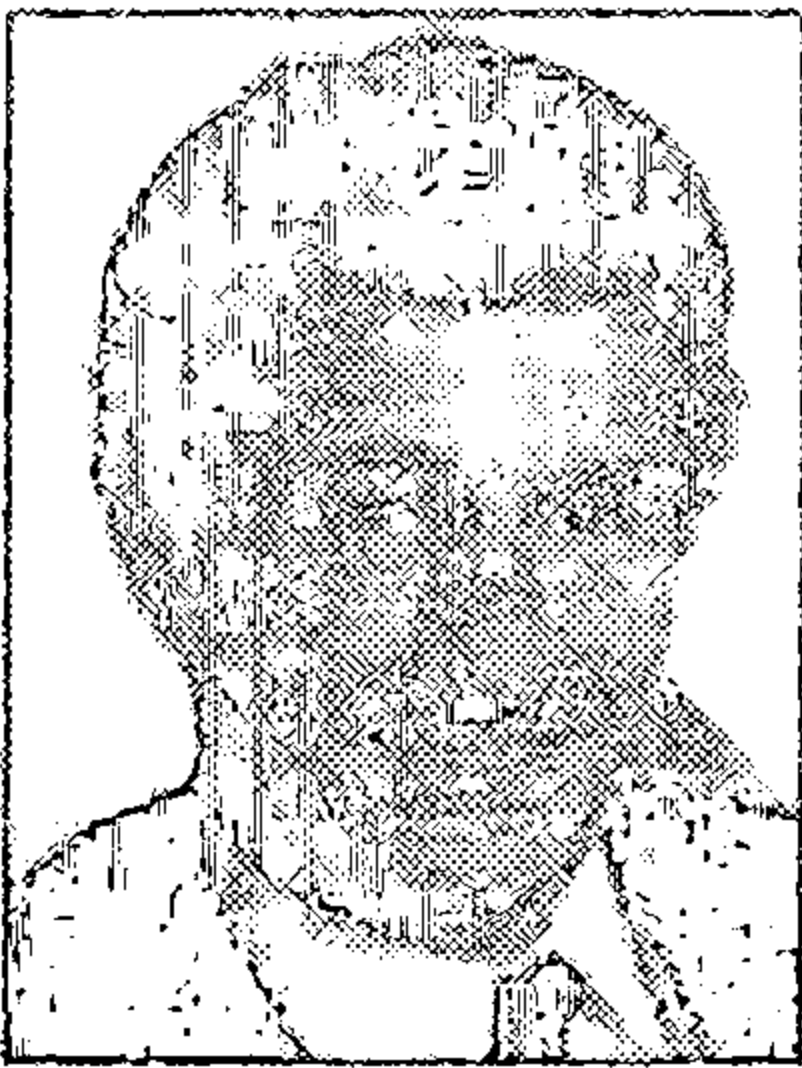


图 1-28 一个 Wireshark 屏幕快照（打印的 Wireshark 屏幕快照得到了 Wireshark 基金会的许可）

你能够在 Web 站点 <http://www.awl.com/kurose-ross> 上找到有关该第一个 Wireshark 实验的全部材料（包括如何获得并安装 Wireshark 的指导）。

人物专访

Leonard Kleinrock 是加州大学洛杉矶分校（UCLA）的计算机科学教授。1969 年，他在 UCLA 的计算机成为因特网的第一个结点。1961 年，他创造的分组交换原理成为因特网的支撑技术。他在纽约城市大学（City College of New York, CCNY）获得电子工程学士学位，并在麻省理工学院（MIT）获得电子工程硕士和博士学位。



Leonard Kleinrock

• 是什么使得您决定专门研究网络/因特网技术的？

当我于 1959 年在 MIT 读博士时，我发现周围的大多数同学正在信息理论和编码理论领域做研究。在 MIT，那时有伟大的研究者 Claude Shannon，他已经开创这些领域，并且已经解决了许多重要的问题。留下来的研究问题既难又不太重要。因此我决定开始新的研究领域，而该领域其他人还没有想到。回想那时在 MIT 我的周围有许多计算机，我很清楚很快这些计算机将有相互通信的需求。在那时，却没有有效的办法来做到这一点，因此我决定研发能够创建有效的数据网络的技术。

• 您在计算机产业的第一份工作是什么？它使您有哪些收益？

1951~1957 年，我为了获得电子工程学士学位在 CCNY 读夜大。在那段时间里，我在一家称为 Photobell 的工业电子小公司工作，先是当技术员，然后当工程师。在那里，我在它们的产品线上引入了数字技术。我们主要使用光电子设备来检测某些物体（盒子、人等）的存在，一种称为双稳态多频振荡器的电路的使用正是我们需要的技术类型，它能将数字处理引入检测领域。这些电路恰好是计算机的基

本模块，用今天的话说就是触发电路或交换器。

- 当您发送第一个主机到主机报文（从 UCLA 到斯坦福研究院）时，您心中想到了什么？

坦率地说，我们当时并没有想到那件事的重要性。我们没有准备具有历史意义的豪言壮语，就像昔日许多发明家所做的那样（如塞缪尔·莫尔斯的“上帝创造了什么（What hath God wrought）”，亚历山大·格瑞汉姆·贝尔的“Watson 先生，请来这里！我想见你”，或尼尔·阿姆斯特朗的“个人的一小步，人类的一大步”）。多么聪明的人哪！他们明白媒体和公众的关系。我们要做的所有工作是向斯坦福研究院的计算机进行注册。当我们键入“L”，它被正确收到，当我们键入“o”又被正确收到，而当我们键入“g”则引起斯坦福研究院主机的崩溃！因此，这将我们的报文转换为最短的，也许是最有预测性的报文，“Lo!”即为“真想不到（Lo and behold）!”。

那年早些时候，UCLA 新闻稿引用我的话说，一旦该网络建立并运行起来，将可能从我们的家中和办公室访问计算机设施，就像我们获得电力和电话连接那样容易。因此那时我的美好愿望是，因特网将是一个无所不在的、总是运行的、总是可用的网络，任何人从任何地方用任何设备将能够与之相连，并且它将是不可见的。然而，我从没有期待我的 99 岁的母亲将能够上因特网，但她确实做到了这一点。

- 您对未来网络的展望是什么？

我的展望中最容易的部分是预测基础设施本身。我预见我们看到移动计算、移动设备和智能空间的大量部署。轻量级、廉价、高性能、便携的计算和通信设备（加上因特网的无处不在）的确使我们成为游牧一员。游牧计算是指使从一个地方旅行到另一个地方的端用户，以透明方式访问因特网服务，无论他们旅行到何处，无论他们携带什么设备或获得何种接入。展望中最困难的部分是预测应用和服务，它们以引人注目的方式不断地带给我们惊喜（电子邮件、搜索技术、万维网、博客、社交网络、用户一代以及音乐、照片和视频等的共享）。我们正面临一种新的惊奇和创新，即移动应用装载于手持设备中。

下一步将使我们从信息空间虚拟世界（netherworld）移动到智能空间的物理世界。我们的环境（办公桌、墙壁、车辆、钟表、腰带等）将因技术而栩栩如生，这些技术包括激励器（actuator）、传感器、逻辑、处理、存储、照相机、麦克风、话筒、显示器和通信。这种嵌入式技术将使得环境能提供我们需要的 IP 服务。当我走进一间房间时，该房间知道我的到来。我将能够与环境自然地通信，如同说英语一样；我的请求产生的响应将从墙上的显示器通过我的眼镜以 Web 网页的形式呈现给我，就像说话、全息照相等一样。

再向前看一点，我看到未来的网络包括了下列附加的关键组件。我看到在网络各处部署的智能软件代理，它的功能是挖掘数据，根据数据采取动作，观察趋势，并能动态地、自适应地执行任务。我看到相当多的网络流量并不是由人产生的，而是由这些嵌入式设备和这些智能软件代理产生的。我看到大批的自组织系统控制这个巨大、快速的网络。我看到巨量的信息瞬间通过网络立即得到强力处理和过滤。因特网最终将是一个无所不在的全球性神经系统。当我们急速在 21 世纪进发时，我将看到这些东西和更多的东西。

- 哪些人激发了您的专业灵感？

到目前为止，是麻省理工学院的 Claude Shannon。他是一名卓越的研究者，具有以高度直觉的方式将他的数学理念与物理世界关联起来的能力。他是我的博士论文答辩委员会的成员。

- 您对进入网络/因特网领域的学生们有什么忠告吗？

因特网和由它使能的所有东西是一个巨大的新前沿，充满了令人惊奇的挑战，为众多创新提供了广阔空间。不要受今天技术的束缚，开动大脑，想象能够做些什么，并去实现它。

应用层

网络应用是计算机网络存在的理由，如果我们不能构想出任何有用的应用，也就没有任何必要去设计支持它们的网络协议了。自因特网发明以来，的确已开发出众多有用的、有趣的网络应用。这些应用程序已经成为因特网成功的驱动力，激励人们在家庭、学校、政府和商业中利用网络，使因特网成为他们日常活动的密不可分的一部分。

因特网应用包括 20 世纪 70 年代和 80 年代开始流行的、经典的基于文本的应用，如文本电子邮件、远程访问计算机、文件传输和新闻组；还包括 20 世纪 90 年代中期的招人喜爱的应用——万维网，包括 Web 冲浪、搜索和电子商务；还包括 20 世纪末引入的两个招人喜爱的应用——即即时讯息和对等（P2P）文件共享。自 2000 年以来，我们见证了流行的语音和视频应用的爆炸，包括 IP 电话（VoIP）、IP 视频会议（如 Skype）；用户生成的视频分布（如 YouTube）；以及点播电影（如 Netflix）。与此同时，我们也看到了极有吸引力的多方在线游戏的出现，包括《第二人生》（Second Life）和《魔兽世界》（World of Warcraft）。最近，我们已经看到了新一代社交网络应用如 Facebook 和 Twitter，它们在因特网的路由器和通信链路网络上创建了引人入胜的人的网络。显然，新型和令人兴奋的因特网应用并没有减缓。也许本书的一些读者将创建下一代招人喜爱的因特网应用。

在本章中，我们学习有关网络应用的原理和实现方面的知识。我们从定义几个关键的应用层概念开始，其中包括应用程序所需要的网络服务、客户和服务端、进程和运输层接口。我们详细考察几种网络应用程序，包括 Web、电子邮件、DNS 和对等文件分发（第 8 章关注多媒体应用，包括流式视频和 IP 电话）。然后我们将涉及开发运行在 TCP 和 UDP 上的网络应用程序。特别是，我们学习套接字 API，并浮光掠影地学习用 Python 语言写的一些简单的客户 - 服务器应用程序。在本章后面，我们也将提供几个有趣、有意思的套接字编程作业。

应用层是我们学习协议非常好的起点，它最为我们所熟悉。我们熟悉的很多应用就是建立在这些将要学习的协议基础上的。通过对应用层的学习，将有助于我们认知协议有关知识，将使我们了解到很多问题，这些问题当我们学习运输层、网络层及数据链路层协议时也同样会碰到。

2.1 应用层协议原理

假定你对新型网络应用有了一些想法。也许这种应用将为人类提供一种伟大的服务，或者将使你的教授高兴，或者将带给你大量的财富，或者只是在开发中获得乐趣。无论你的动机是什么，我们现在考察一下如何将你的想法转变为一种真实世界的网络应用。

研发网络应用程序的核心是写出能够运行在不同的端系统和通过网络彼此通信的程序。例如，在 Web 应用程序中，有两个互相通信的不同的程序：一个是运行在用户主机（桌面机、膝上机、平板电脑、智能电话等）上的浏览器程序；另一个是运行在 Web 服务器主机上的 Web 服务器程序。另一个例子是 P2P 文件共享系统，在参与文件共享的社区

中的每台主机中都有一个程序。在这种情况下，在各台主机中的这些程序可能都是类似的或相同的。

因此，当研发新应用程序时，你需要编写将在多台端系统上运行的软件。例如，该软件能够用 C、Java 或 Python 来编写。重要的是，你不需要写在网络核心设备如路由器或链路层交换机上运行的软件。即使你要为网络核心设备写应用程序软件，你也不能做到这一点。如我们在第 1 章所知，以及如图 1-24 所显示的那样，网络核心设备并不在应用层上起作用，而仅在较低层起作用，特别是位于网络层及下面层次。这种基本设计，也即将应用软件限制在端系统（如图 2-1 所示）的方法，促进了大量的网络应用程序的迅速研发和部署。

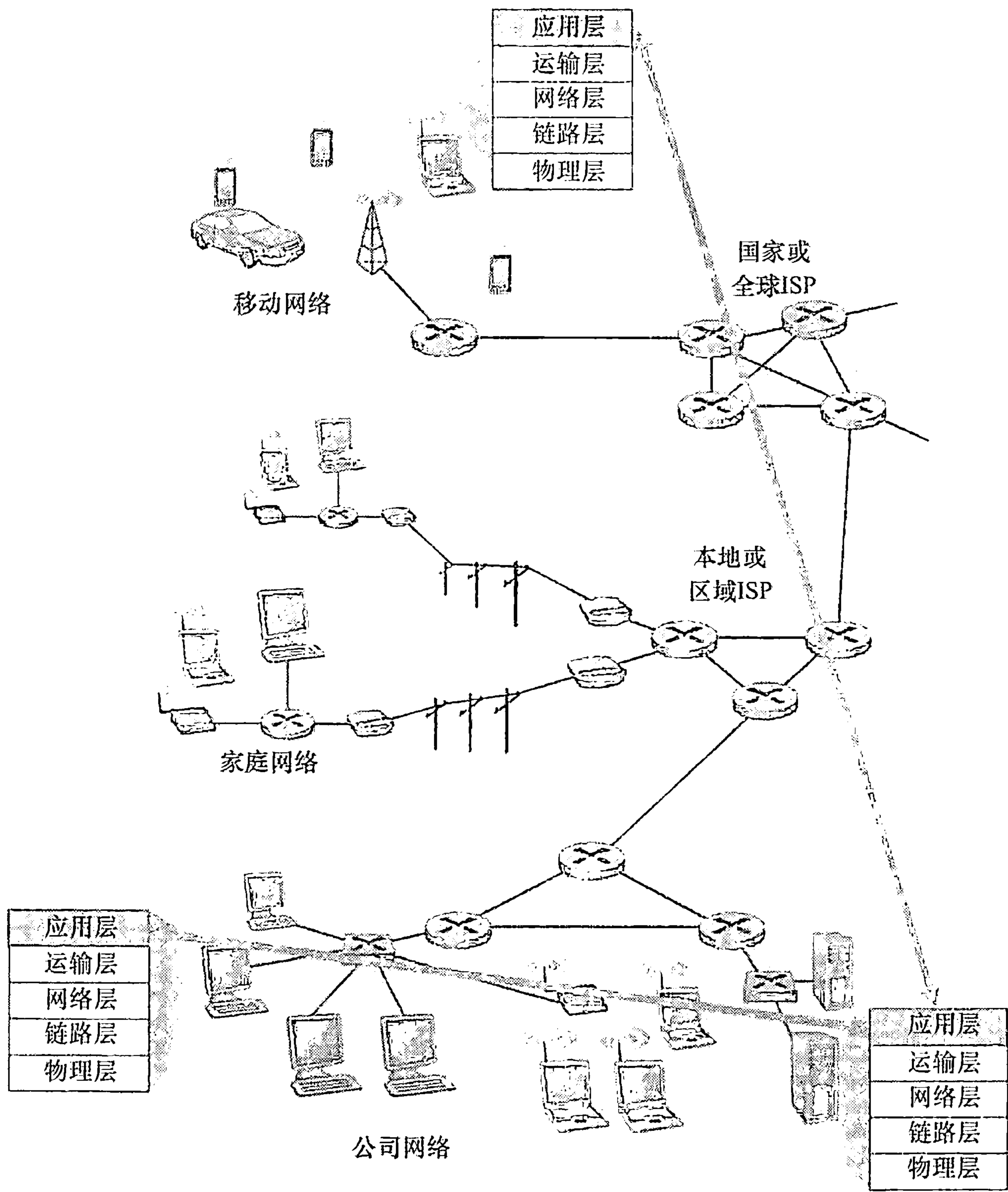


图 2-1 在应用层的端系统之间的网络应用的通信

2. 1. 1 网络应用程序体系结构

当进行软件编码之前，应当对应用程序有一个宽泛的体系结构计划。记住应用程序的体系结构明显不同于网络的体系结构（例如在第 1 章中所讨论的 5 层因特网体系结构）。

从应用程序研发者的角度看，网络体系结构是固定的，并为应用程序提供了特定的服务集合。在另一方面，应用程序体系结构（application architecture）由应用程序研发者设计，规定了如何在各种端系统上组织该应用程序。在选择应用程序体系结构时，应用程序研发者很可能利用现代网络应用程序中所使用的两种主流体系结构之一：客户-服务器体系结构或对等（P2P）体系结构。

在客户-服务器体系结构（client-server architecture）中，有一个总是打开的主机称为服务器，它服务于来自许多其他称为客户的主机的请求。一个经典的例子是 Web 应用程序，其中总是打开的 Web 服务器服务于来自浏览器（运行在客户主机上）的请求。当 Web 服务器接收到来自某客户对某对象的请求时，它向该客户发送所请求的对象作为响应。值得注意的是利用客户-服务器体系结构，客户相互之间不直接通信；例如，在 Web 应用中两个浏览器并不直接通信。客户-服务器体系结构的另一个特征是该服务器具有固定的、周知的地址，该地址称为 IP 地址（我们将很快讨论它）。因为该服务器具有固定的、周知的地址，并且因为该服务器总是打开的，客户总是能够通过向该服务器的 IP 地址发送分组来与其联系。具有客户-服务器体系结构的非常著名的应用程序包括 Web、FTP、Telnet 和电子邮件。图 2-2a 中显示了这种客户-服务器体系结构。

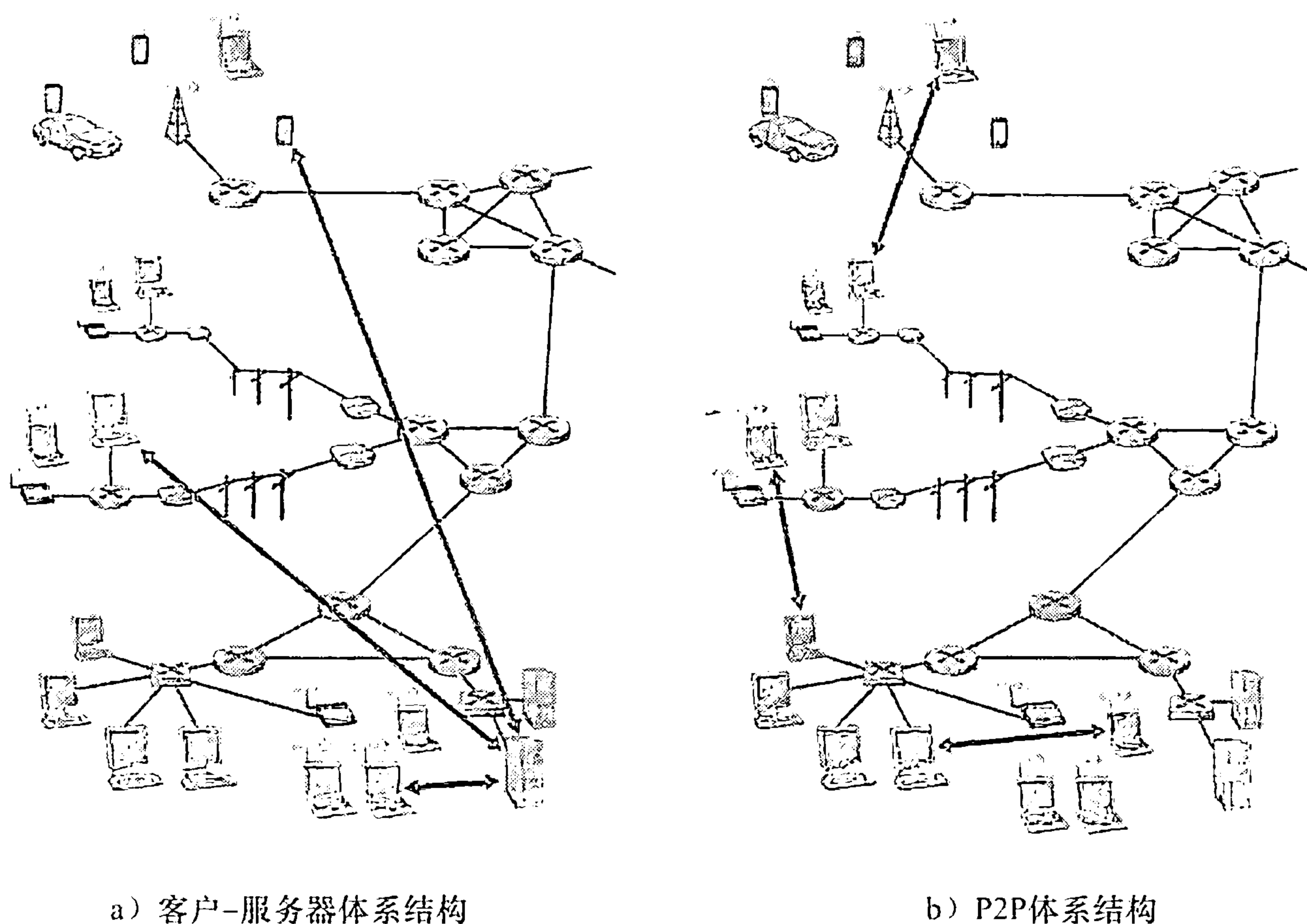


图 2-2 客户-服务器体系结构及 P2P 体系结构

在一个客户-服务器应用中，常常会出现一台单独的服务器主机跟不上它所有客户请求的情况。例如，一个流行的社交网络站点如果仅有一台服务器来处理所有请求，将很快变得不堪重负。为此，配备大量主机的数据中心常被用于创建强大的虚拟服务器。最为流行的因特网服务——如搜索引擎（如谷歌和 Bing）、因特网商务（如亚马逊和 e-Bay）、基于 Web 的电子邮件（如 Gmail 和雅虎邮件）、社交网络（如脸谱和推特），就应用了一个或多个数据中心。如在 1.3.3 节中所讨论的那样，谷歌有分布于全世界的 30 ~ 50 个数据中心，这些数据中心共同处理搜索、YouTube、Gmail 和其他服务。一个数据中心能够有数十万台服务器，它们必须要供电和维护。此外，服务提供商必须支付不断出现的互联和带

宽费用，以发送和接收到达/来自数据中心的数据。

在一个 P2P 体系结构（P2P architecture）中，对位于数据中心的专用服务器有最小的（或者没有）依赖。相反，应用程序在间断连接的主机对之间使用直接通信，这些主机对被称为对等方。这些对等方并不为服务提供商所有，相反却为用户控制的桌面机和膝上机所有，大多数对等方驻留在家庭、大学和办公室。因为这种对等方通信不必通过专门的服务器，该体系结构被称为对等方到对等方的。许多目前流行的、流量密集型应用都是 P2P 体系结构的。这些应用包括文件共享（例如 BitTorrent）、对等方协助下载加速器（例如迅雷）、因特网电话（例如 Skype）和 IPTV（例如“迅雷看看”和 PPstream）。图 2-2b 中显示了 P2P 的体系结构。需要提及的是，某些应用具有混合的体系结构，它结合了客户 - 服务器和 P2P 的元素。例如，对于许多即时讯息应用而言，服务器被用于跟踪用户的 IP 地址，但用户到用户的报文在用户主机之间（无需通过中间服务器）直接发送。

P2P 体系结构的最引人入胜的特性之一是它们的自扩展性（self-scalability）。例如，在一个 P2P 文件共享应用中，尽管每个对等方都由于请求文件产生工作量，但每个对等方通过向其他对等方分发文件也为系统增加服务能力。P2P 体系结构也是成本有效的，因为它们通常不需要庞大的服务器基础设施和服务器带宽（这与具有数据中心的客户 - 服务器设计形成反差）。然而，未来 P2P 应用面临三个主要挑战：

- ISP 友好。大多数住宅 ISP（包括 DSL 和电缆 ISP）已经受制于“非对称的”带宽应用，也就是说，下载比上载要多得多。但是 P2P 视频流和文件分发应用改变了从服务器到住宅 ISP 的上载流量，因而给 ISP 带来了巨大压力。未来 P2P 应用需要设计对 ISP 友好的模式 [Xie 2008]。
- 安全性。因为它们的高度分布和开放特性，P2P 应用给安全带来挑战 [Douceur 2002; Yu 2006; Liang 2006; Naoumov 2006; Dhungel 2008; LeBlond 2011]。
- 激励。未来 P2P 应用的成功也取决于说服用户自愿向应用提供带宽、存储和计算资源，这对激励设计带来挑战 [Feldman 2005; Piatek 2008; Aperjis 2008; Liu 2010]。

2.1.2 进程通信

在构建网络应用程序前，还需要对运行在多个端系统上的程序是如何互相通信的情况有一个基本了解。在操作系统的术语中，进行通信的实际上是进程（process）而不是程序。一个进程可以被认为是在运行在端系统中的一个程序。当进程运行在相同的端系统上时，它们使用进程间通信机制相互通信。进程间通信的规则由端系统上的操作系统确定。而在本书中，我们不怎么关注同一台主机上的进程间的通信，而关注运行在不同端系统（可能具有不同的操作系统）上的进程间的通信。

在两个不同端系统上的进程，通过跨越计算机网络交换报文（message）而相互通信。发送进程生成并向网络中发送报文；接收进程接收这些报文并可能通过将报文发送回去进行响应。图 2-1 图示了进程是如何通过使用 5 层协议栈的应用层互相通信的。

1. 客户和服务进程

网络应用程序由成对的进程组成，这些进程通过网络相互发送报文。例如，在 Web

应用程序中，一个客户浏览器进程与一台 Web 服务器进程交换报文。在一个 P2P 文件共享系统中，文件从一个对等方中的进程传输到另一个对等方中的进程。对每对通信进程，我们通常将这两个进程之一标识为客户（client），而另一个进程标识为服务器（server）。对于 Web 而言，浏览器是一个客户进程，Web 服务器是一台服务器进程。对于 P2P 文件共享，下载文件的对等方标识为客户，上载文件的对等方标识为服务器。

你或许已经观察到，如在 P2P 文件共享的某些应用中，一个进程能够既是客户又是服务器。在 P2P 文件共享系统中，一个进程的确既能上载文件又能下载文件。无论如何，在任何给定的一对进程之间的通信会话场景中，我们仍能将一个进程标识为客户，另一个进程标识为服务器。我们定义客户和服务器进程如下：

在给定的一对进程之间的通信会话场景中，发起通信（即在该会话开始时发起与其他进程的联系）的进程被标识为客户，在会话开始时等待联系的进程是服务器。

在 Web 中，一个浏览器进程向一台 Web 服务器进程发起联系，因此该浏览器进程是客户，而该 Web 服务器进程是服务器。在 P2P 文件共享中，当对等方 A 请求对等方 B 发送一个特定的文件时，在这个特定的通信会话中对等方 A 是客户，而对等方 B 是服务器。在不致混淆的情况下，我们有时也使用术语“应用程序的客户端和服务端”。在本章的结尾，我们将逐步讲解网络应用程序的客户端和服务端的简单代码。

2. 进程与计算机网络之间的接口

如上所述，多数应用程序是由通信进程对组成，每对中的两个进程互相发送报文。从一个进程向另一个进程发送的报文必须通过下面的网络。进程通过一个称为套接字（socket）的软件接口向网络发送报文和从网络接收报文。我们考虑一个类比来帮助我们理解进程和套接字。进程可类比于一座房子，而它的套接字可以类比于它的门。当一个进程想向位于另外一台主机上的另一个进程发送报文时，它把报文推出该门（套接字）。该发送进程假定该门到另外一侧之间有运输的基础设施，该设施将把报文传送到目的进程的门口。一旦该报文抵达目的主机，它通过接收进程的门（套接字）传递，然后接收进程对该报文进行处理。

图 2-3 显示了两个经过因特网通信的进程之间的套接字通信（图 2-3 中假定由该进程使用的下面运输层协议是因特网的 TCP 协议）。如该图所示，套接字是同一台主机内应用层与运输层之间的接口。由于该套接字是建立网络应用程序的可编程接口，因此套接字也称为应用程序和网络之间的应用程序编程接口（Application Programming Interface, API）。应用程序开发者可以控制套接字在应用层端的一切，但是对该套接字的运输层端几乎没有控制权。应用程序开发者对于运输层的控制仅限于：①选择运输层协议；②也许能设定几个运输层参数，如最大缓存和最大报文段长度等（将在第 3 章中涉及）。一旦应用程序开发者选择了一个运输层协议（如果可供选择的话），则应用程序就建立在由该协议提供的运输层服务之上。我们将在 2.7 节中对套接字进行更为详细的探讨。

3. 进程寻址

为了向特定目的地发送邮政邮件，目的地需要有一个地址。类似地，在一台主机上运行的进程为了向在另一台主机上运行的进程发送分组，接收进程需要有一个地址。为了标识该接收进程，需要定义两种信息：①主机的地址；②定义在目的主机中的接收进程的标识符。

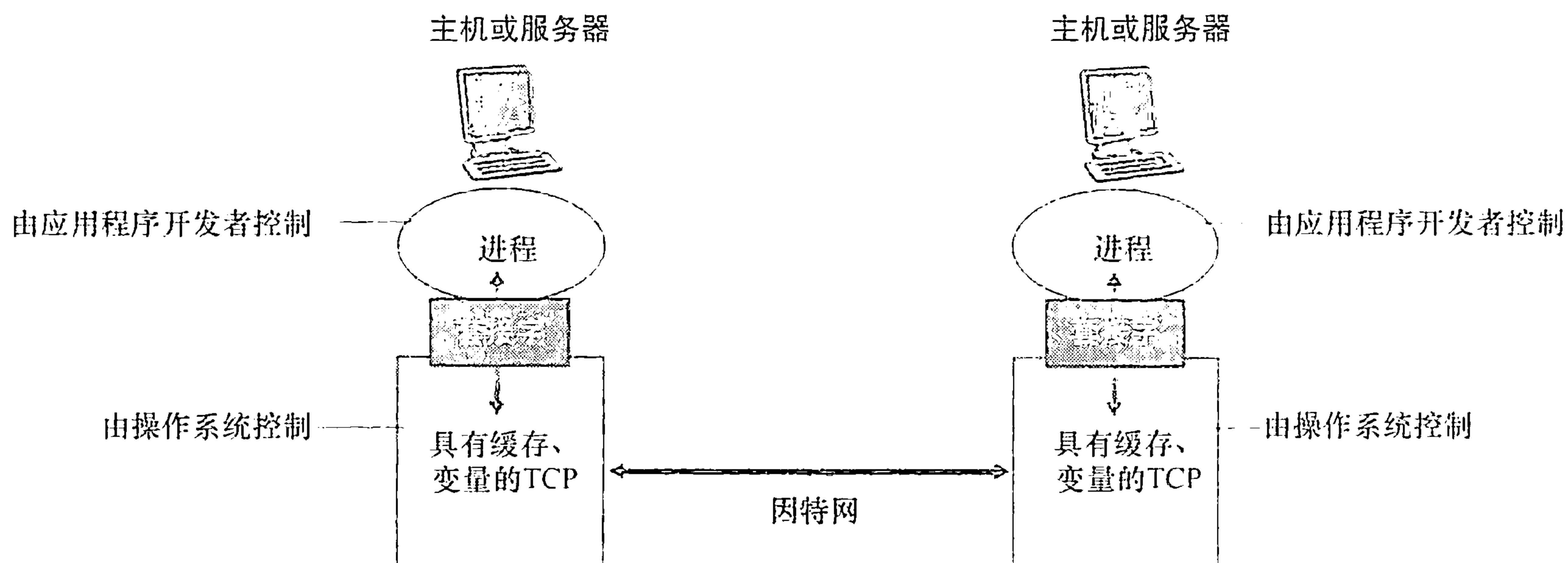


图 2-3 应用进程、套接字和下面的运输层协议

在因特网中，主机由其 IP 地址（IP address）标识。我们将在第 4 章中非常详细地讨论 IP 地址。此时，我们只要知道 IP 地址是一个 32 比特的量且它能够唯一地标识该主机就够了。除了知道报文送往目的地的主机地址外，发送进程还必须指定运行在接收主机上的接收进程（更具体地说，接收套接字）。因为一般而言一台主机能够运行许多网络应用，这些信息是需要的。目的地端口号（port number）用于这个目的。已经给流行的应用分配了特定的端口号。例如，Web 服务器用端口号 80 来标识。邮件服务器进程（使用 SMTP 协议）用端口号 25 来标识。用于所有因特网标准协议的周知端口号的列表能够在 <http://www.iana.org> 处找到。我们将在第 3 章中详细学习端口号。

2.1.3 可供应用程序使用的运输服务

前面讲过套接字是应用程序进程和运输层协议之间的接口。在发送端的应用程序将报文推进该套接字。在该套接字的另一侧，运输层协议负责使该报文进入接收进程的套接字。

包括因特网在内的很多网络提供了不止一种运输层协议。当开发一个应用时，必须选择一种可用的运输层协议。如何做出这种选择呢？最可能的方式是，通过研究这些可用的运输层协议所提供的服务，选择一个最能为你的应用需求提供恰当服务的协议。这种情况类似于在两个城市间旅行时选择飞机还是火车作为交通工具。你必须选择一种或另一种，而且每种运输模式为你提供不同的服务（例如，火车可以直到市区上客和下客，而飞机提供了更短的旅行时间）。

一个运输层协议能够为调用它的应用程序提供什么样的服务呢？我们大体能够从四个方面对应用程序服务要求进行分类：可靠数据传输、吞吐量、定时和安全性。

1. 可靠数据传输

如第 1 章讨论的那样，分组在计算机网络中可能丢失。例如，分组能够使路由器中的缓存溢出，或者当分组中的某些比特损坏后可能被丢弃。像电子邮件、文件传输、远程主机访问、Web 文档传输以及金融应用等这样的应用，数据丢失可能会造成灾难性的后果（在后一种情况下，无论对银行或对顾客都是如此！）。因此，为了支持这些应用，必须做一些工作以确保由应用程序的一端发送的数据正确、完全地交付给该应用程序的另一端。